

Intermediate Level (Laboratorios)

Módulo. Acceso a datos y ORM

Laboratorio:

Modelos User/Order/OrderItem, CRUD básico, migración Alembic y pruebas en SQLite en memoria

Se creará un módulo completo que incluya:

- Modelos ORM con SQLAlchemy 2.0:
 - User (1:N) Order
 - Order (1:N) OrderItem
 - Control de Transacciones y Operaciones CRUD.
 - Gestión de migraciones con Alembic.
 - Pruebas unitarias e integración con pytest usando SQLite en memoria
-
1. Configurar dependencias, antes de escribir los modelos, nos aseguramos de que la carpeta `intermediate_level` tenga instaladas las librerías necesarias (sqlalchemy, alembic, pytest, etc)



2. Configuración del Engine / Session:
 - Configuración de la base de datos con SQLAlchemy 2.0.

```
from collections.abc import Generator

from sqlalchemy import create_engine
from sqlalchemy.orm import DeclarativeBase, Session, sessionmaker

# URL para desarrollo / pruebas locales con SQLite
DATABASE_URL = "sqlite:///./app.db"

# Engine de SQLAlchemy 2.0
engine = create_engine(
    DATABASE_URL,
    echo=False, # Cambiar a True para ver el SQL generado durante
    depuración
```

```

)

# Fábrica de sesiones
SessionLocal = sessionmaker(
    bind=engine,
    autocommit=False,
    autoflush=False,
    expire_on_commit=False,
)

# Clase Base Declarativa para los modelos ORM
class Base(DeclarativeBase):
    pass

# Generador / Inyector de sesiones
def get_db() -> Generator[Session, None, None]:
    """Provee una sesión de base de datos dentro de un bloque de
    contexto."""
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

```

3. Crear los modelos en src/modulo01/models.py

En este archivo se definirán tres entidades y sus relaciones 1:N:

- Un User tiene muchos Order (pedidos)
- Un order pertenece a un User y tiene muchos OrderItem (Items del pedido).
- Un OrderItem representa el detalle de un producto en la orden.

```

from decimal import Decimal

from sqlalchemy import ForeignKey, Numeric, String
from sqlalchemy.orm import Mapped, mapped_column, relationship

from .database import Base

```

```

class User(Base):
    __tablename__ = "users"

    id: Mapped[int] = mapped_column(primary_key=True)
    username: Mapped[str] = mapped_column(String(50), unique=True,
nullable=False)
    email: Mapped[str] = mapped_column(String(120), unique=True,
nullable=False)

    # Relación 1:N hacia Order
    orders: Mapped[list["Order"]] = relationship(
        back_populates="user", cascade="all, delete-orphan"
    )

    def __repr__(self) -> str:
        return f"<User(id={self.id}, username='{self.username}',
email='{self.email}')>"

class Order(Base):
    __tablename__ = "orders"

    id: Mapped[int] = mapped_column(primary_key=True)
    user_id: Mapped[int] = mapped_column(
        ForeignKey("users.id", ondelete="CASCADE"), nullable=False
    )
    status: Mapped[str] = mapped_column(String(20), default="PENDING",
nullable=False)

    # Relación inversa hacia User
    user: Mapped["User"] = relationship(back_populates="orders")

    # Relación 1:N hacia OrderItem
    items: Mapped[list["OrderItem"]] = relationship(
        back_populates="order", cascade="all, delete-orphan"
    )

    def __repr__(self) -> str:
        return f"<Order(id={self.id}, user_id={self.user_id},
status='{self.status}')>"

```

```

class OrderItem(Base):
    __tablename__ = "order_items"

    id: Mapped[int] = mapped_column(primary_key=True)
    order_id: Mapped[int] = mapped_column(
        ForeignKey("orders.id", ondelete="CASCADE"), nullable=False
    )
    product_name: Mapped[str] = mapped_column(String(100),
    nullable=False)
    quantity: Mapped[int] = mapped_column(nullable=False, default=1)
    price: Mapped[Decimal] = mapped_column(Numeric(10, 2),
    nullable=False)

    # Relación inversa hacia Order
    order: Mapped["Order"] = relationship(back_populates="items")

    def __repr__(self) -> str:
        return (
            f"<OrderItem(id={self.id}, product='{self.product_name}', "
            f"qty={self.quantity}, price={self.price})>"
        )

```

4. Implementación de Repositorio CRUD (src/modulo1/repository.py)

En este caso se contruira la capa de acceso a datos aplicando patrones de SQLAlchemy 2.0:

- Sentencias select() explícitas (en lugar de Query de SQLAlchemy 1.x).
- Manejo explícito de la sesión para operaciones de pesistencia.
- Manejo de trasacción atómicas usando bloques db.begin() para asegurar consistencia (por ejemplo, si falla la inserción de un item en una orden, se revierte toda la transacción)

```

from collections.abc import Sequence
from decimal import Decimal

from sqlalchemy import select
from sqlalchemy.orm import Session

from .models import Order, OrderItem, User

```

```

class UserRepository:
    """Repositorio para operaciones CRUD de la entidad User."""

    def __init__(self, db: Session) -> None:
        self.db = db

    def create_user(self, username: str, email: str) -> User:
        """Crea y persiste un nuevo usuario."""
        user = User(username=username, email=email)
        self.db.add(user)
        self.db.commit()
        self.db.refresh(user)
        return user

    def get_by_id(self, user_id: int) -> User | None:
        """Obtiene un usuario por su ID primario."""
        return self.db.get(User, user_id)

    def get_by_username(self, username: str) -> User | None:
        """Obtiene un usuario filtrando por su nombre de usuario."""
        stmt = select(User).where(User.username == username)
        return self.db.scalars(stmt).first()

    def get_all(self) -> Sequence[User]:
        """Retorna todos los usuarios registrados."""
        stmt = select(User)
        return self.db.scalars(stmt).all()

    def delete_user(self, user_id: int) -> bool:
        """Elimina un usuario por ID. Retorna True si existía y se
eliminó."""
        user = self.get_by_id(user_id)
        if not user:
            return False
        self.db.delete(user)
        self.db.commit()
        return True

class OrderRepository:

```

```

    """Repositorio para operaciones transaccionales de Order y
    OrderItem."""

    def __init__(self, db: Session) -> None:
        self.db = db

    def create_order_with_items(
        self, user_id: int, items_data: list[tuple[str, int, Decimal]]
    ) -> Order:
        """
        Crea una orden junto con sus ítems en una sola transacción
        atómica.
        items_data es una lista de tuplas: (product_name, quantity,
        price)
        """
        # Creación de la orden principal
        order = Order(user_id=user_id, status="PENDING")

        # Asociación de ítems a la relación 1:N
        for product_name, quantity, price in items_data:
            item = OrderItem(
                product_name=product_name,
                quantity=quantity,
                price=price,
            )
            order.items.append(item)

        # Persistencia en base de datos
        self.db.add(order)
        self.db.commit()
        self.db.refresh(order)
        return order

    def get_order_by_id(self, order_id: int) -> Order | None:
        """Obtiene una orden por su ID."""
        return self.db.get(Order, order_id)

    def update_order_status(self, order_id: int, new_status: str) ->
    Order | None:
        """Actualiza el estado de una orden existente."""
        order = self.get_order_by_id(order_id)
        if not order:

```

```

        return None
    order.status = new_status
    self.db.commit()
    self.db.refresh(order)
    return order

```

- Verificación del paso 4: Una vez guardado el archivo, ejecuta en la terminal los formateadores y linters.

5. Configuración de Migraciones con Alembic:

Alembic es la herramienta oficial de migraciones de SQLAlchemy. Nos permite rastrear cambios en el esquema de la base de datos (creación de tablas, nuevas columnas, índices) a través de scripts versionados sin perder datos.

- Inicializar el entorno de Alembic:

```

$ poetry run alembic init alembic
Creating directory C:\Users\juan.rivera\Documents\Curso_Python\intermediate_level\alembic\versions ... done
Generating C:\Users\juan.rivera\Documents\Curso_Python\intermediate_level\alembic.ini ... done
Generating C:\Users\juan.rivera\Documents\Curso_Python\intermediate_level\alembic\env.py ... done
Generating C:\Users\juan.rivera\Documents\Curso_Python\intermediate_level\alembic\README ... done
Generating C:\Users\juan.rivera\Documents\Curso_Python\intermediate_level\alembic\script.py.mako ... done
Please edit configuration/connection/logging settings in C:\Users\juan.rivera\Documents\Curso_Python\intermediate_level\alembic.ini before proceeding.
(intermediate-level-py3.13)

```

- Configurar alembic/env.py
Necesitamos conectar Alembic con nuestra Base Declarativa (Base) para que detecte automáticamente los modelos User, Order y OrderItem

- Añadimos la importación de nuestro "Base":

```

from logging.config import fileConfig

import src.modulo01.models # noqa: F401
from alembic import context
from sqlalchemy import engine_from_config, pool
from src.modulo01.database import Base

```

- Buscamos la línea que dice target_metadata = None y la cambiamos por:

```

21 target_metadata = Base.metadata

```

- Nos aseguramos de que de la URL de la base de datos en alembic.ini apunte a SQLite local.

```

86 # database URL. This is consumed by the
87 # other means of configuring database URL
88 # file.
89 sqlalchemy.url = sqlite:///./app.db

```

- Generar la primera migración automática, una vez configurado alembic/env.py y alembic.ini, generara el primer script de migración ejecutanto:

```

juan.rivera@mxjuriagdell5400 MINGW64 C:/Users/juan.rivera/Documents/Curso_Python/intermediate_level (feature/modulo-01-acceso-datos-orm)
$ poetry run alembic revision --autogenerate -m "Crear tablas iniciales users orders order_items"
INFO [alembic.runtime.migration] Context impl SQLiteImpl.
INFO [alembic.runtime.migration] Will assume non-transactional DDL.
INFO [alembic.runtime.plugins] setting up autogenerate plugin alembic.autogenerate.schemas
INFO [alembic.runtime.plugins] setting up autogenerate plugin alembic.autogenerate.tables
INFO [alembic.runtime.plugins] setting up autogenerate plugin alembic.autogenerate.types
INFO [alembic.runtime.plugins] setting up autogenerate plugin alembic.autogenerate.constraints
INFO [alembic.runtime.plugins] setting up autogenerate plugin alembic.autogenerate.defaults
INFO [alembic.runtime.plugins] setting up autogenerate plugin alembic.autogenerate.comments
INFO [alembic.autogenerate.compare.tables] Detected added table 'users'
INFO [alembic.autogenerate.compare.tables] Detected added table 'orders'
INFO [alembic.autogenerate.compare.tables] Detected added table 'order_items'
Generating C:\Users\juan.rivera\Documents\Curso_Python\intermediate_level\alembic\versions\eeefd1cb1d158_crear_tablas_iniciales_users_orders_.py ... done
(intermediate-level-py3.13)

```

- Aplicar la migración a la base de datos:

```

juan.rivera@mxjuriagdell5400 MINGW64 C:/Users/juan.rivera/Documents/Curso_Python/intermediate_level (feature/modulo-01-acceso-datos-orm)
$ poetry run alembic upgrade head
INFO [alembic.runtime.migration] Context impl SQLiteImpl.
INFO [alembic.runtime.migration] Will assume non-transactional DDL.
INFO [alembic.runtime.migration] Running upgrade -> eeefd1cb1d158, Crear tablas iniciales users orders order_items
(intermediate-level-py3.13)

```

- Ejecutamos las pruebas unitarias con reporte de cobertura:

```

juan.rivera@mxjuriagdell5400 MINGW64 C:/Users/juan.rivera/Documents/Curso_Python/intermediate_level (feature/modulo-01-acceso-datos-orm)
$ poetry run pytest --cov=src
===== test session starts =====
platform win32 -- Python 3.13.15, pytest-9.1.1, pluggy-1.6.0
rootdir: C:\Users\juan.rivera\Documents\Curso_Python\intermediate_level
configfile: pyproject.toml
plugins: anyio-4.15.1, cov-7.1.0
collected 3 items

tests\test_modulo1.py ... [100%]

===== tests coverage =====
_____ coverage: platform win32, python 3.13.15-final-0 _____

Name                                Stmts   Miss  Cover
-----
src\intermediate_level\__init__.py      0      0  100%
src\modulo01\__init__.py                 0      0  100%
src\modulo01\database.py                13      4   69%
src\modulo01\models.py                  31      3   90%
src\modulo01\repository.py              51     11   78%
-----
TOTAL                                  95     18   81%

===== 3 passed in 0.68s =====
(intermediate-level-py3.13)

```

Módulo. APIs web con FastAPI (Automatización)

Laboratorio:

CRUD de Orders con validación; login JWT básico; tests de integración con DB temporal

Alcance del laboratorio:

- CRUD de Orderd con validación meidante Pydnatic y SQLAlchemy.
- Login JWT básico para autenticación y protección de endpoints.
- Test de integración con un base datos temporal (SQLite en memoria) para validar la API.

1. Instalación de dependencias para el Módulo 02:

Nos aseguramos de estar ubicados en la carpeta `intermediate_level/` y ejecutamos el siguiente comando para agregar FastAPI, Unicorn, componentes de seguridad y el cliente HTTP para pruebas.

```
juan.rivera@mxjuriagdel15400 MINGW64 C:/Users/juan.rivera/Documents/Curso_Python/intermediate_level (feature/modulo-02-APIs-Web-FastAPI)
$ poetry add "fastapi[standard]" pydantic-settings email-validator python-jose[cryptography] passlib[bcrypt] httpx
The following packages are already present in the pyproject.toml and will be skipped:

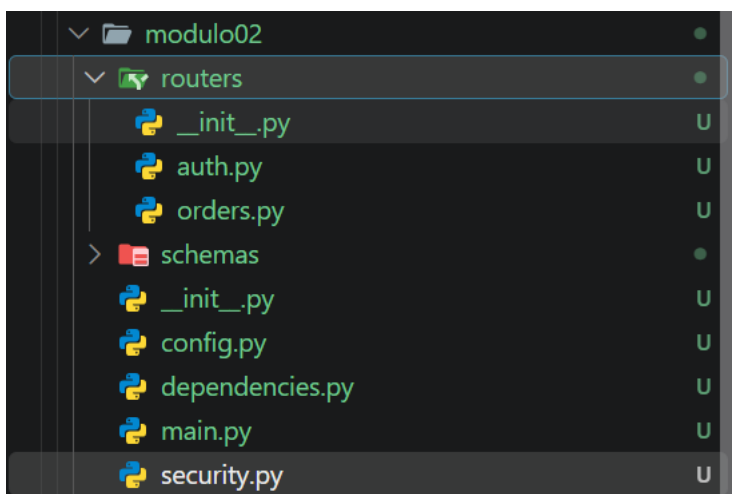
- httpx

If you want to update it to the latest compatible version, you can use `poetry update package`.
If you prefer to upgrade it to the latest available version, you can use `poetry add package@latest`.

Using version ^0.141.1 for fastapi
Using version ^2.15.0 for pydantic-settings
Using version ^2.3.0 for email-validator
Using version ^3.5.0 for python-jose
Using version ^1.7.4 for passlib

Updating dependencies
Resolving dependencies... Downloading https://files.pythonhosted.org/packages/68/69/854f072529f9e0ed2cc97b81adc5f20a18dab60bc84dd0eec7446d2166
Resolving dependencies... (9.1s)
```

2. Crear la estructura de carpetas y archivos base:



3. Configuración general (src/modulo02/config.py):

Abrimos el archivo `src/modulo02/config.py` e implementamos la lectura de configuraciones mediante Pydantic Settings.

```
config.py U X
intermediate_level > src > modulo02 > config.py > Settings
1  from pydantic_settings import BaseSettings, SettingsConfigDict
2
3
4  class Settings(BaseSettings):
5      PROJECT_NAME: str = "FastAPI Orders API"
6      VERSION: str = "1.0.0"
7      API_V1_STR: str = "/api/v1"
8
9      # Configuración de JWT
10     SECRET_KEY: str = (
11         "09d25e094faa6ca2556c818166b7a9563b93f7099f6f0f4caa6cf63b88e8d3e7"
12     )
13     ALGORITHM: str = "HS256"
14     ACCESS_TOKEN_EXPIRE_MINUTES: int = 60
15
16     model_config = SettingsConfigDict(
17         env_file=".env", case_sensitive=True, extra="ignore"
18     )
19
20
21     Settings = Settings()
```

4. Definir los esquemas dePydantic

Creamos los modelos de Pydantic para validar las peticiones HTTP y dar formato a las respuestas JSON.

- Esquema de autenticación (`src/modulo02/schemas/auth.py`)

```
auth.py U X  order.py U X
intermediate_level > src > modulo02 > schemas > auth.py > Token
1  from pydantic import BaseModel, Field
2
3
4  class UserLogin(BaseModel):
5      username: str = Field(..., min_length=3, max_length=50)
6      password: str = Field(..., min_length=6)
7
8  class Token(BaseModel):
9      access_token: str
10     token_type: str = "bearer"
11
12  class TokenData(BaseModel):
13     username: str | None = None
14
15
```

- Esquema de órdenes e ítems (src/modulo02/schemas/order.py)

```
auth.py 2, U • order.py U X
intermediate_level > src > modulo02 > schemas > order.py > ...
1  from decimal import Decimal
2
3  from pydantic import BaseModel, ConfigDict, Field
4
5
6  class OrderItemBase(BaseModel):
7      product_name: str = Field(..., min_length=1, max_length=100)
8      quantity: int = Field(..., gt=0)
9      price: Decimal = Field(..., gt=0)
10
11
12  class OrderItemCreate(OrderItemBase):
13      pass
14
15
16  class OrderItemResponse(OrderItemBase):
17      id: int
18      order_id: int
19
20      model_config = ConfigDict(from_attributes=True)
21
22
23  class OrderCreate(BaseModel):
24      id: int
25      user_id: int
26      status: str
27      items: list[OrderItemResponse] = []
28
29      model_config = ConfigDict(from_attributes=True)
```

5. Seguridad y JWT (src/modulo02/security.py)

Aquí implementamos las funciones utilitarias para:

- Generar la clase (hash) de contraseñas
- Verificar contraseñas al iniciar sesión
- Crar token de acceso JWT firmados con el algoritmo HS256

```
from datetime import datetime, timedelta, timezone
```

```

from jose import jwt
from passlib.context import CryptContext

from .config import settings

# Configuración del contexto de encriptación bcrypt para contraseñas
pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")

def verify_password(plain_password: str, hashed_password: str) -> bool:
    """Verifica que la contraseña plana coincida con el hash
    almacenado."""
    return pwd_context.verify(plain_password, hashed_password)

def get_password_hash(password: str) -> str:
    """Genera un hash bcrypt a partir de una contraseña plana."""
    return pwd_context.hash(password)

def create_access_token(
    data: dict, expires_delta: timedelta | None = None
) -> str:
    """Crea un token de acceso JWT firmado."""
    to_encode = data.copy()

    if expires_delta:
        expire = datetime.now(timezone.utc) + expires_delta
    else:
        expire = datetime.now(timezone.utc) + timedelta(
            minutes=settings.ACCESS_TOKEN_EXPIRE_MINUTES
        )

    to_encode.update({"exp": expire})
    encoded_jwt = jwt.encode(
        to_encode, settings.SECRET_KEY, algorithm=settings.ALGORITHM
    )
    return encoded_jwt

```

6. Inyección de dependencias (src/modulo02/dependencias.py)

- La sesión de base de datos de SQLAlchemy (reutilizando el motor de Módulo 01).
- La extracción y validación del token JWT para proteger endpoints.

```
from collections.abc import Generator

from fastapi import Depends, HTTPException, status
from fastapi.security import OAuth2PasswordBearer
from jose import JWTError, jwt
from sqlalchemy.orm import Session

# Reutilizamos la sesión del Módulo 01 e importaciones relativas del
# Módulo 02
from ..modulo01.database import SessionLocal
from .config import settings
from .schemas.auth import TokenData

# Esquema OAuth2 que buscará el token en el encabezado Authorization:
# Bearer <token>
oauth2_scheme =
OAuth2PasswordBearer(tokenUrl=f"{settings.API_V1_STR}/auth/login")

def get_db() -> Generator[Session, None, None]:
    """Inyecta una sesión de base de datos SQLAlchemy por request."""
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

def get_current_user(token: str = Depends(oauth2_scheme)) -> TokenData:
    """Valida el token JWT y retorna los datos del usuario
    autenticado."""
    credentials_exception = HTTPException(
        status_code=status.HTTP_401_UNAUTHORIZED,
        detail="No se pudieron validar las credenciales",
        headers={"WWW-Authenticate": "Bearer"},
    )
    try:
```

```

    payload = jwt.decode(
        token, settings.SECRET_KEY, algorithms=[settings.ALGORITHM]
    )
    username: str | None = payload.get("sub")
    if username is None:
        raise credentials_exception
    return TokenData(username=username)
except JWTError:
    raise credentials_exception

```

7. Crear los Routers de Autenticación y Ordenes:

En este paso conectaremos las rutas de la API, aplicando las reglas de negocio, los esquemas de Pydantic y las dependencias creadas.

- Router de Autenticación (src/modulo02/routers/auth.py), este endpoint valida las credenciales recibidas via OAuth2PasswordRequestForm o JSON Y retorna un token JWT firmado.

```

from typing import Annotated

from fastapi import APIRouter, Depends, HTTPException, status
from fastapi.security import OAuth2PasswordRequestForm
from sqlalchemy.orm import Session

from ...modulo01.models import User
from ..dependencies import get_db
from ..schemas.auth import Token
from ..security import create_access_token, verify_password

router = APIRouter(prefix="/auth", tags=["Auth"])

@router.post("/login", response_model=Token)
def login_for_access_token(
    form_data: Annotated[OAuth2PasswordRequestForm, Depends()],
    db: Annotated[Session, Depends(get_db)],
):
    """Endpoint de autenticación para obtener el token JWT."""
    user = (

```

```

        db.query(User).filter(User.username ==
form_data.username).first()
    )

    # Nota: Si el password almacenado aún no tiene hash en la DB de
prueba,
    # se valida en texto plano como respaldo o con verify_password.
    if not user or not (
        verify_password(form_data.password, user.password_hash)
        or user.password_hash == form_data.password
    ):
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Nombre de usuario o contraseña incorrectos",
            headers={"WWW-Authenticate": "Bearer"},
        )

    access_token = create_access_token(data={"sub": user.username})
    return Token(access_token=access_token, token_type="bearer")

```

- Router del CRUD de Órdenes (src/modulo02/routers/orders.py), definimos los endpoints del CRUD con protección vía token JWT.

```

from typing import Annotated

from fastapi import APIRouter, Depends, HTTPException, status
from sqlalchemy.orm import Session

from ...modulo01.models import Order, OrderItem, User
from ..dependencies import get_current_user, get_db
from ..schemas.auth import TokenData
from ..schemas.order import OrderCreate, OrderResponse

router = APIRouter(prefix="/orders", tags=["Orders"])

@router.post("/", response_model=OrderResponse,
status_code=status.HTTP_201_CREATED)
def create_order(
    order_in: OrderCreate,

```

```

    db: Annotated[Session, Depends(get_db)],
    current_user: Annotated[TokenData, Depends(get_current_user)],
):
    """Crea una nueva orden asociada a un usuario existente."""
    user = db.query(User).filter(User.id == order_in.user_id).first()
    if not user:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail=f"El usuario con ID {order_in.user_id} no existe",
        )

    new_order = Order(user_id=order_in.user_id, status="PENDING")
    db.add(new_order)
    db.flush() # Asigna ID a la orden antes de insertar los ítems

    for item in order_in.items:
        new_item = OrderItem(
            order_id=new_order.id,
            product_name=item.product_name,
            quantity=item.quantity,
            price=item.price,
        )
        db.add(new_item)

    db.commit()
    db.refresh(new_order)
    return new_order

@router.get("/{order_id}", response_model=OrderResponse)
def get_order(
    order_id: int,
    db: Annotated[Session, Depends(get_db)],
    current_user: Annotated[TokenData, Depends(get_current_user)],
):
    """Obtiene el detalle de una orden por su ID."""
    order = db.query(Order).filter(Order.id == order_id).first()
    if not order:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail=f"Orden con ID {order_id} no encontrada",
        )

```

```

        return order

@router.get("/", response_model=list[OrderResponse])
def list_orders(
    db: Annotated[Session, Depends(get_db)],
    current_user: Annotated[TokenData, Depends(get_current_user)],
    skip: int = 0,
    limit: int = 10,
):
    """Lista las órdenes de la base de datos de forma paginada."""
    return db.query(Order).offset(skip).limit(limit).all()

```

8. Ensamblar la Aplicación Principal (src/modulo02/main.py)

Abrimos el archivo src/modulo02/main.py. Aquí ensamblaremos la aplicación de FastAPI, agregaremos la configuración de CORS y registraremos los enrutadores de autenticación y órdenes con su prefijo común (/api/v1).

```

from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware

from .config import settings
from .routers import auth, orders

app = FastAPI(
    title=settings.PROJECT_NAME,
    version=settings.VERSION,
    openapi_url=f"{settings.API_V1_STR}/openapi.json",
    docs_url="/docs",
    redoc_url="/redoc",
)

# Configuración del Middleware de CORS
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

```

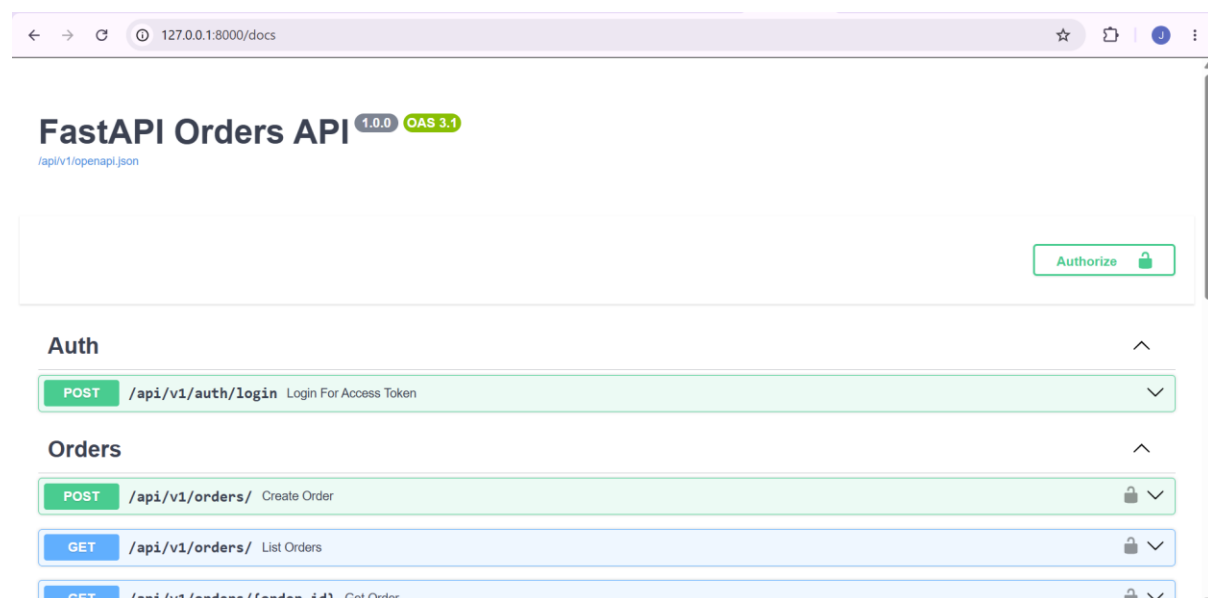
```
# Registro de Routers
app.include_router(auth.router, prefix=settings.API_V1_STR)
app.include_router(orders.router, prefix=settings.API_V1_STR)

@app.get("/health", tags=["Health Check"])
def health_check() -> dict[str, str]:
    """Endpoint de verificación de estado de la API."""
    return {"status": "ok", "version": settings.VERSION}
```

9. Iniciar y probar el servidor manualmente

Para validar que la aplicación levante sin errores de sintaxis o de importación, ejecutamos desde la carpeta `intermediate_level/`

```
juan.rivera@mxjuniagdel15400 MINGW64 C:/Users/juan.rivera/Documents/Curso_Python/intermediate_level (feature/modulo-02-APIs-Web-FastAPI)
$ poetry run uvicorn src.modulo02.main:app --reload
INFO: Will watch for changes in these directories: ['C:\\Users\\juan.rivera\\Documents\\Curso_Python\\intermediate_level']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [24092] using WatchFiles
INFO: Started server process [23528]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: 127.0.0.1:60536 - "GET / HTTP/1.1" 404 Not Found
INFO: 127.0.0.1:60536 - "GET /favicon.ico HTTP/1.1" 404 Not Found
INFO: 127.0.0.1:62526 - "GET /docs HTTP/1.1" 200 OK
INFO: 127.0.0.1:62526 - "GET /api/v1/openapi.json HTTP/1.1" 200 OK
```



10. Pruebas de Integración (pytest + httpx)

Para cumplir con el objetivo del laboratorio sin afectar la base de datos real, crearemos una suite de pruebas que utiliza una base de datos SQLite en memoria durante la ejecución de los tests.

- Escribir los tests de integración (tests/test_modulo02.py), incluimos el setup con fixtures de pytest, la sobrescritura de dependencias de la base de datos (app.dependency_overrides) y la validación de los endpoints:

```
from collections.abc import Generator

import pytest
from fastapi.testclient import TestClient
from sqlalchemy import create_engine
from sqlalchemy.orm import Session, sessionmaker
from sqlalchemy.pool import StaticPool
from src.modulo01.database import Base
from src.modulo01.models import User
from src.modulo02.dependencies import get_db
from src.modulo02.main import app
from src.modulo02.security import get_password_hash

# Motor SQLite en memoria aislado para pruebas
SQLALCHEMY_DATABASE_URL = "sqlite:///memory:"

engine = create_engine(
    SQLALCHEMY_DATABASE_URL,
    connect_args={"check_same_thread": False},
    poolclass=StaticPool,
)

TestingSessionLocal = sessionmaker(
    autocommit=False, autoflush=False, bind=engine
)

@pytest.fixture(scope="function")
def db_session() -> Generator[Session, None, None]:
    """Crea las tablas en la BD en memoria antes de cada test y las
    elimina al finalizar."""
    Base.metadata.create_all(bind=engine)
    session = TestingSessionLocal()
    try:
```

```

        yield session
    finally:
        session.close()
        Base.metadata.drop_all(bind=engine)

@pytest.fixture(scope="function")
def client(db_session: Session) -> Generator[TestClient, None, None]:
    """Cliente HTTP de prueba que reemplaza la sesión de BD de
    producción por la de prueba."""

    def override_get_db():
        try:
            yield db_session
        finally:
            pass

    app.dependency_overrides[get_db] = override_get_db
    with TestClient(app) as test_client:
        yield test_client
    app.dependency_overrides.clear()

def test_health_check(client: TestClient):
    """Verifica que el endpoint de salud devuelva 200 OK."""
    response = client.get("/health")
    assert response.status_code == 200
    assert response.json()["status"] == "ok"

def test_login_and_create_order_flow(client: TestClient, db_session:
Session):
    """Prueba el flujo completo: Crear usuario -> Login JWT -> Crear
    Orden Protegida."""
    # 1. Crear un usuario de prueba directamente en la BD temporal
    hashed_password = get_password_hash("password123")
    test_user = User(
        username="juan_tester",
        email="juan@example.com",
        password_hash=hashed_password,
    )
    db_session.add(test_user)

```

```

db_session.commit()
db_session.refresh(test_user)

# 2. Hacer Login para obtener el Token JWT
login_data = {"username": "juan_tester", "password": "password123"}
login_response = client.post("/api/v1/auth/login", data=login_data)
assert login_response.status_code == 200
token = login_response.json()["access_token"]
assert token is not None

# 3. Intentar crear una orden SIN token (debe retornar 401
Unauthorized)
order_payload = {
    "user_id": test_user.id,
    "items": [
        {"product_name": "Teclado Mecánico", "quantity": 1,
"price": 120.50}
    ],
}
unauthorized_response = client.post("/api/v1/orders/",
json=order_payload)
assert unauthorized_response.status_code == 401

# 4. Crear la orden CON el token Bearer
headers = {"Authorization": f"Bearer {token}"}
authorized_response = client.post(
    "/api/v1/orders/", json=order_payload, headers=headers
)
assert authorized_response.status_code == 201
created_order = authorized_response.json()
assert created_order["user_id"] == test_user.id
assert created_order["status"] == "PENDING"
assert len(created_order["items"]) == 1

# 5. Consultar la orden creada por ID
order_id = created_order["id"]
get_response = client.get(f"/api/v1/orders/{order_id}",
headers=headers)
assert get_response.status_code == 200
assert get_response.json()["id"] == order_id

```

11. Ejecutar las pruebas de integración:

```

$ poetry run pytest tests/test_modulo02.py -v
===== test session starts =====
platform win32 -- Python 3.13.15, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\juan.rivera\AppData\Local\pypoetry\Cache\virtualenvs\intermediate-level-zUiHL2GR-py3.13\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\juan.rivera\Documents\Curso_Python\intermediate_level
configfile: pyproject.toml
plugins: anyio-4.15.1, cov-7.1.0
collected 2 items

tests/test_modulo02.py::test_health_check PASSED [ 50%]
tests/test_modulo02.py::test_login_and_create_order_flow PASSED [100%]

```

Módulo. Pruebas y TDD

Laboratorio:

- Implementar una historia nueva con TDD; añadir test de propiedades; reporte de cobertura

Diseñaremos la clase `PaymentProcessor` que depende de una pasarela de pago externa (`PaymentGateway`).

Historia de Usuario

Como sistema de comercio electrónico, **Quiero** procesar pagos aplicando descuentos por código promocional y manejando reintentos automáticos ante fallos de red, **Para** asegurar cobros confiables y notificar al usuario.

Reglas de Negocio a Implementas:

- Cálculo de Descuentos (`apply_discount`):
 - Soporta código “PROMO10” (10% de descuento) y “PROMO20” (20% de descuento).
 - Cualquier otro código lanza un “InvalidCouponError”
 - El monto final con descuento no puede ser menor a 0
- Procesamiento con Reintentos (`process_payment`):
 - Debe llamar a la pasarela de pago `PaymentGateway.charge(amount, cad_number)`.
 - Si la pasarela lanza una excepción de red (`GatewayNetworkError`), el procesador debe reintentar hasta 3 veces.
 - Si tras el tercer intento sigue fallando, lanza `PaymentFailedError`.
 - Si el cobro es exitoso, devuelve `True`.

1. Configurar la estructura del laboratorio

- Creamos el archivo de la pasarela simulada `src/modulo03/gatewal.py`. Este archivo definirá la interfaz externa con la que interactuaremos:

```
gateway.py U X
intermediate_level > src > modulo03 > gateway.py > ...
1  #src/modulo03/gateway.py
2
3  class GatewayNetworkError(Exception):
4      """Excepción lanzada cuando hay un fallo de red o tiempo de espera en la pasarela"""
5
6
7  class PaymentGateway:
8      def charge(self, amount: float, card_number: str) -> bool:
9          # En producción, esto se conecta a Stripe/Paypal
10         return True
11
```

2. Fase RED para Regla 1 (Cálculo de Descuentos)

- Creamos el archivo de pruebas tests/test_payment_processor.py con los primeros casos de prueba para el método apply_discount:

```
gateway.py U X  test_payment_processor.py 1, U ●
intermediate_level > tests > test_payment_processor.py > ...
1  # tests/test_payment_processor.py
2  import pytest
3
4  # Intentamos importar componentes que aún no existe en el procesador
5  from src.modulo03.processor import (
6      InvalidCouponError,
7      PaymentProcessor,
8  )
9
10
11  def test_apply_discount_valid_promo10():
12      processor = PaymentProcessor()
13      final_price = processor.apply_discount(amount=100.0, coupon="PROMO10")
14      assert final_price == 90.0
15
16  def test_apply_discount_valid_promo20():
17      processor = PaymentProcessor()
18      final_price = processor.apply_discount(amount=200.0, coupon="PROMO20")
19      assert final_price == 160.0
20
21  def test_apply_discount_invalid_coupon_raises_exception():
22      processor = PaymentProcessor()
23      with pytest.raises(InvalidCouponError):
24          processor.apply_discount(amount=100.0, coupon="INVALIDO")
```

- Confirmamos que obtenemos el error RED esperado:

```

juan.rivera@mxjuriagdel15400 MINGW64 C:/Users/juan.rivera/Documents/Curso_Python/intermediate_level (feature/modulo-03-pruebas-tdd)
$ poetry run pytest tests/test_payment_processor.py -v
rootdir: C:\Users\juan.rivera\Documents\Curso_Python\intermediate_level
configfile: pyproject.toml
plugins: anyio-4.15.1, hypothesis-6.168.1, cov-7.1.0
collected 0 items / 1 error

===== ERRORS =====
_____ ERROR collecting tests/test_payment_processor.py _____
ImportError while importing test module 'C:\Users\juan.rivera\Documents\Curso_Python\intermediate_level\tests\test_payment_processor.py'.
Hint: make sure your test modules/packages have valid Python names.
Traceback:
..\..\..\AppData\Local\Programs\Python\Python313\Lib\importlib\__init__.py:88: in import_module
    return _bootstrap._gcd_import(name[level:], package, level)
tests\test_payment_processor.py:5: in <module>
    from src.modulo03.processor import (
E   ModuleNotFoundError: No module named 'src.modulo03.processor'
===== short test summary info =====
ERROR tests/test_payment_processor.py
!!!!!!!!!!!! Interrupted: 1 error during collection !!!!!!!!!!!!!!!
===== 1 error in 0.51s =====
(intermediate-level-py3.13)

```

3. Fase GREEN para Regla 1 (Cálculo de Descuentos)

- Implementaremos el código mínimo indispensable en `src/modulo03/processor.py` para cumplir con las reglas de negocio de los descuentos y hacer que las pruebas pasen.

```

gateway.py x test_payment_processor.py x processor.py 1, x
intermediate_level > src > modulo03 > processor.py > ...
1  # src/modulo03/processor.py
2
3  class InvalidCouponError(Exception):
4      """Excepción lanzada cuando el código promocional ingresado no es válido."""
5
6
7
8  class PaymentProcessor:
9
10     DISCOUNTS = {
11         "PROMO10": 0.10,
12         "PROMO20": 0.20,
13     }
14
15     def apply_discount(self, amount: float, coupon: str | None = None) -> float:
16         if not coupon:
17             return amount
18
19         coupon_upper = coupon.upper()
20         if coupon_upper not in self.DISCOUNTS:
21             raise InvalidCouponError(f"El cupón '{coupon}' no es válido.")
22
23         discount_rate = self.DISCOUNTS[coupon_upper]
24         discounted_amount = amount * (1 - discount_rate)
25         return max(0.0, discounted_amount)

```

- Ejecutamos pruebas y confirmamos que las tres pruebas de descuentos pasen a GREEN (PASSED).

```

juan.rivera@mxjuniagdel15400 MINGW64 C:/Users/juan.rivera/Documents/Curso_Python/intermediate_level (feature/modulo-03-pruebas-tdd)
$ poetry run pytest tests/test_payment_processor.py -v
===== test session starts =====
platform win32 -- Python 3.13.15, pytest-9.1.1, pluggy-1.6.0 -- C:/Users/juan.rivera/AppData/Local/pypoetry\Cache/virtualenvs/intermediate-level-zUjHL2GR-py3.
13/Scripts/python.exe
cachedir: .pytest_cache
hypothesis profile 'default'
rootdir: C:/Users/juan.rivera/Documents/Curso_Python/intermediate_level
configfile: pyproject.toml
plugins: anyio-4.15.1, hypothesis-6.168.1, cov-7.1.0
collected 3 items

tests/test_payment_processor.py::test_apply_discount_valid_promo10 PASSED [ 33%]
tests/test_payment_processor.py::test_apply_discount_valid_promo20 PASSED [ 66%]
tests/test_payment_processor.py::test_apply_discount_invalid_coupon_raises_exception PASSED [100%]

===== 3 passed in 0.26s =====
(intermediate-level-py3.13)

```

4. Poetry-Based Test para descuentos (Hypothesis)

Antes de pasar al procesamiento con reintentos y mocks, agregamos una prueba basada en propiedades con Hypothesis para garantizar la siguiente propiedad del sistema:

Invariante del Descuento:

Para cualquier monto positivo $A > 0$ y para cualquier cupón válido (“PROMO10” o “PROMO20”, el precio final F siempre debe ser menor que el monto original ($F < A$) y nunca puede ser menor a cero ($F \geq 0$).

- Agregamos el Property Test a tests/test_payment_processor.py :

```

29 # --- Property-Based Test para Descuentos ---|
30 @given(
31     amount=st.floats(min_value=1.0, max_value=1e6, allow_nan=False, allow_infinity=False),
32     coupon=st.sampled_from(["PROMO10", "PROMO20"]),
33 )
34 def test_property_discount_always_reduces_price_and_is_non_negative(
35     amount: float, coupon: str
36 ):
37     processor = PaymentProcessor()
38     discounted = processor.apply_discount(amount=amount, coupon=coupon)
39
40     # Invariante 1: El precio con descuento debe ser menor al original
41     assert discounted < amount
42     # Invariante 2: El precio resultante no debe ser negativo
43     assert discounted >= 0.0

```

-
- Ejecutamos las pruebas:

```

juan.rivera@mxjuriagdell5400 MINGW64 C:/Users/juan.rivera/Documents/Curso_Python/intermediate_level (feature/modulo-03-pruebas-tdd)
$ poetry run pytest tests/test_payment_processor.py -v
===== test session starts =====
platform win32 -- Python 3.13.15, pytest-9.1.1, pluggy-1.6.0 -- C:/Users/juan.rivera/AppData/Local/pypoetry\Cache/virtualenvs/intermediate-level-zUiHL2GR-py3.13/Scripts/python.exe
cachedir: .pytest_cache
hypothesis profile 'default'
rootdir: C:/Users/juan.rivera/Documents/Curso_Python/intermediate_level
configfile: pyproject.toml
plugins: anyio-4.15.1, hypothesis-6.168.1, cov-7.1.0
collected 4 items

tests/test_payment_processor.py::test_apply_discount_valid_promo10 PASSED [ 25%]
tests/test_payment_processor.py::test_apply_discount_valid_promo20 PASSED [ 50%]
tests/test_payment_processor.py::test_apply_discount_invalid_coupon_raises_exception PASSED [ 75%]
tests/test_payment_processor.py::test_property_discount_always_reduces_price_and_is_non_negative PASSED [100%]

===== 4 passed in 0.47s =====
(intermediate-level-py3.13)

```

5. Fase RED para la regla 2 (Procesamiento con Mocks y Reintentos)

Ahora implementamos la segunda regla de negocio: procesar un pago usando la pasarela PaymentGateway con hasta 3 reintentos ante fallos de red (GatewayNetworkError).

Especificaciones de la prueba:

- Éxito al primer intento: Si la pasarela responde True, process_payment retorna True.
 - Éxito tras reintentos: Si la pasarela falla 2 veces con GatewayNetworkError y al 3er intento funciona, la función retorna True (se llamará a charge exactamente 3 veces).
 - Fallo definitivo tras 3 reintentos: Si la pasarela falla las 3 veces seguidas con GatewayNetworkError, el procesador lanza la excepción personalizada PaymentFailedError.
- Agregar los tests con Mocks en tests/test_payment_processor.py:

```

# --- Tests de Procesamiento de Pagos con Mocks y Reintentos ---
def test_process_payment_success_on_first_try():
    mock_gateway = MagicMock(spec=PaymentGateway)
    mock_gateway.charge.return_value = True

    processor = PaymentProcessor(gateway=mock_gateway)
    result = processor.process_payment(amount=100.0, card_number="1234-5678-9012-3456")

    assert result is True
    mock_gateway.charge.assert_called_once_with(100.0, "1234-5678-9012-3456")

```

```

def test_process_payment_retries_and_succeeds_on_third_try():
    mock_gateway = MagicMock(spec=PaymentGateway)
    # Falla 2 veces por red y la 3ra funciona
    mock_gateway.charge.side_effect = [
        GatewayNetworkError("Error de red 1"),
        GatewayNetworkError("Error de red 2"),
        True,
    ]

    processor = PaymentProcessor(gateway=mock_gateway)
    result = processor.process_payment(amount=100.0, card_number="1234-
5678-9012-3456")

    assert result is True
    assert mock_gateway.charge.call_count == 3

def test_process_payment_fails_after_max_retries_raises_exception():
    mock_gateway = MagicMock(spec=PaymentGateway)
    # Falla siempre por red
    mock_gateway.charge.side_effect = GatewayNetworkError("Fallo
constante")

    processor = PaymentProcessor(gateway=mock_gateway)

    with pytest.raises(PaymentFailedError):
        processor.process_payment(amount=100.0, card_number="1234-5678-
9012-3456")

    assert mock_gateway.charge.call_count == 3

```

- Ejecutamos las pruebas:

```
$ poetry run pytest tests/test_payment_processor.py -v
platform win32 -- Python 3.13.15, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\juan.rivera\AppData\Local\py poetry\Cache\virtualenvs\intermediate-level-zUiHL2GR-py3.13\Scripts\python.exe
cachedir: .pytest_cache
hypothesis profile 'default'
rootdir: C:\Users\juan.rivera\Documents\Curso_Python\intermediate_level
configfile: pyproject.toml
plugins: anyio-4.15.1, hypothesis-6.168.1, cov-7.1.0
collected 0 items / 1 error

===== ERRORS =====
_____ ERROR collecting tests/test_payment_processor.py _____
ImportError while importing test module 'C:\Users\juan.rivera\Documents\Curso_Python\intermediate_level\tests\test_payment_processor.py'.
Hint: make sure your test modules/packages have valid Python names.
Traceback:
..\..\AppData\Local\Programs\Python\Python313\Lib\importlib\__init__.py:88: in import_module
    return _bootstrap._gcd_import(name[level:], package, level)
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
tests\test_payment_processor.py:10: in <module>
    from src.modulo03.processor import (
E ImportError: cannot import name 'PaymentFailedError' from 'src.modulo03.processor' (C:\Users\juan.rivera\Documents\Curso_Python\intermediate_level\src\modulo03\processor.py)
===== short test summary info =====
ERROR tests/test_payment_processor.py
!!!!!!!!!!!! Interrupted: 1 error during collection !!!!!!!!!!!!!
===== 1 error in 0.66s =====
(intermediate-level-py3.13)
```

6. Fase GREEN para la regla 2 (Lógica de Reintentos y Mocks)

Ahora implementaremos la clase `PaymentFailedError` y el método `process_payment` con la lógica de reintentos en `src/modulo03/processor.py`.

- Actualizar `src/modulo03/processor.py`:

```
# src/modulo03/processor.py

from src.modulo03.gateway import GatewayNetworkError, PaymentGateway

class InvalidCouponError(Exception):
    """Excepción lanzada cuando el código promocional ingresado no es válido."""

class PaymentFailedError(Exception):
    """Excepción lanzada cuando el pago falla tras agotar todos los reintentos."""

class PaymentProcessor:

    DISCOUNTS = {
        "PROMO10": 0.10,
```

```

        "PROMO20": 0.20,
    }

    def __init__(self, gateway: PaymentGateway | None = None,
max_retries: int = 3) -> None:
        self.gateway = gateway or PaymentGateway()
        self.max_retries = max_retries

    def apply_discount(self, amount: float, coupon: str | None = None)
-> float:
        if not coupon:
            return amount

        coupon_upper = coupon.upper()
        if coupon_upper not in self.DISCOUNTS:
            raise InvalidCouponError(f"El cupón '{coupon}' no es
válido.")

        discount_rate = self.DISCOUNTS[coupon_upper]
        discounted_amount = amount * (1 - discount_rate)
        return max(0.0, discounted_amount)

    def process_payment(
self, amount: float, card_number: str, coupon: str | None =
None
) -> bool:
        final_amount = self.apply_discount(amount, coupon)

        for attempt in range(self.max_retries):
            try:
                return self.gateway.charge(final_amount, card_number)
            except GatewayNetworkError:
                if attempt == self.max_retries - 1:
                    raise PaymentFailedError(
                        f"El pago de {final_amount} falló tras
{self.max_retries} intentos por error de red."
                    )
        return False

```

- Ejecutamos las pruebas:

```

juan.rivera@mxjuriagdell5400 MINGW64 C:/Users/juan.rivera/Documents/Curso_Python/intermediate_level (feature/modulo-03-pruebas-tdd)
$ poetry run pytest tests/test_payment_processor.py -v
===== test session starts =====
platform win32 -- Python 3.13.15, pytest-9.1.1, pluggy-1.6.0 -- C:/Users/juan.rivera/AppData/Local/pypoetry\Cache/virtualenvs\intermediate-level-zUiHL2GR-py3.13\Scripts\python.exe
cachedir: .pytest_cache
hypothesis profile 'default'
rootdir: C:/Users/juan.rivera/Documents/Curso_Python/intermediate_level
configfile: pyproject.toml
plugins: anyio-4.15.1, hypothesis-6.168.1, cov-7.1.0
collected 7 items

tests/test_payment_processor.py::test_apply_discount_valid_promo10 PASSED [ 14%]
tests/test_payment_processor.py::test_apply_discount_valid_promo20 PASSED [ 28%]
tests/test_payment_processor.py::test_apply_discount_invalid_coupon_raises_exception PASSED [ 42%]
tests/test_payment_processor.py::test_property_discount_always_reduces_price_and_is_non_negative PASSED [ 57%]
tests/test_payment_processor.py::test_process_payment_success_on_first_try PASSED [ 71%]
tests/test_payment_processor.py::test_process_payment_retries_and_succeeds_on_third_try PASSED [ 85%]
tests/test_payment_processor.py::test_process_payment_fails_after_max_retries_raises_exception PASSED [100%]

===== 7 passed in 0.62s =====

```

7. Reporte final de cobertura, para terminar, se mide la cobertura del código completa de la suite de pruebas que se construyó:

```

..\\..\\AppData\\Local\\pypoetry\\Cache\\virtualenvs\\intermediate-level-zUiHL2GR-py3.13\\Lib\\site-packages\\fastapi\\testclient.py:1
C:/Users/juan.rivera/AppData/Local/pypoetry\\Cache\\virtualenvs\\intermediate-level-zUiHL2GR-py3.13\\Lib\\site-packages\\fastapi\\testclient.py:1: StarletteDeprecati
onWarning: Using 'httpx' with 'starlette.testclient' is deprecated; install 'httpx2' instead.
  from starlette.testclient import TestClient as TestClient # noqa

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
===== tests coverage =====
_____ coverage: platform win32, python 3.13.15-final-0 _____

Name                               Stmts  Miss  Cover   Missing
-----
src\\modulo03\\__init__.py           0      0  100%
src\\modulo03\\gateway.py            4      0  100%
src\\modulo03\\notifier.py            4      0  100%
src\\modulo03\\processor.py          26      0  100%
src\\modulo03\\wallet.py             24      0  100%
-----
TOTAL                               58      0  100%
===== 33 passed, 1 warning in 3.43s =====
(intermediate-level-py3.13)

```

Módulo. Concurrencia y Rendimiento

Laboratorio:

- Fetcher concurrente con `httpx.AsyncClient` y semáforo; comparación con versión síncrona; pequeño cálculo CPU-bound con `ProcessPoolExecutor`

En este laboratorio se construirá un sistema completo que combina E/S asíncrona con control de concurrencia y procesamiento numérico en paralelo, midiendo además la mejora en rendimiento con respecto a un enfoque síncrono.

Estructura del Laboratorio

Se crea la carpeta `src/modulo04/` y se desarrollaran tres archivos:

- `src/modulo04/sync_fetcher.py`: Descargador de datos síncrono tradicional (baseline)
- `src/modulo04/async_fetcher.py`: Descargador de datos asíncrono con `httpx.AsyncClient` y un `asyncio.Semaphore` para limitar peticiones concurrentes.
- `src/modulo04/processor.py`: Módulo con una función CPU-bound (procesamiento intensivo de datos) ejecutada mediante `ProcessPoolExecutor`.

1. Configuración de Dependencias en Poetry

Necesitamos la librería httpx (estándar moderno para peticiones HTTP tanto síncronas como asíncronas), verificamos que la tengamos descargada:

```
juan.rivera@mxjuriagdell5400 MINGW64 C:/Users/juan.rivera/Documents/Curso_Python/intermediate_level (feature/modulo-03-pruebas-tdd)
$ poetry add httpx
The following packages are already present in the pyproject.toml and will be skipped:

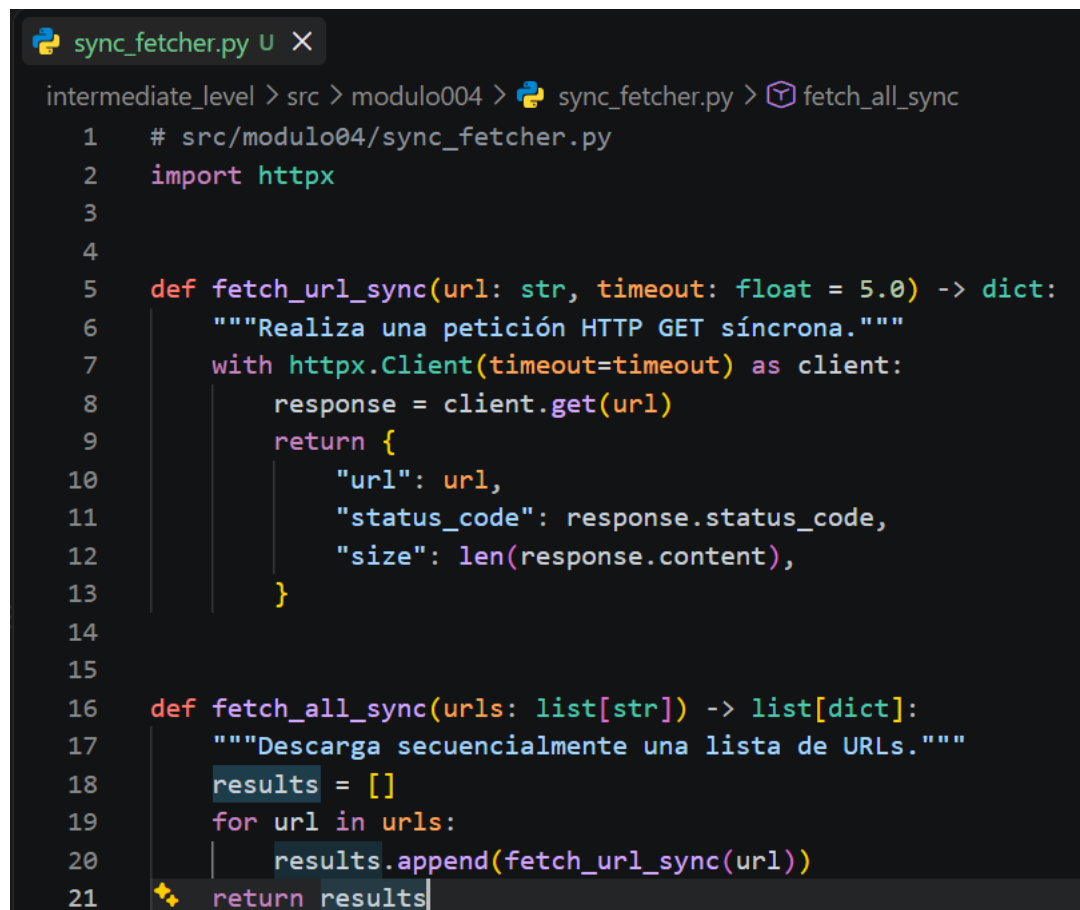
- httpx

If you want to update it to the latest compatible version, you can use `poetry update package`.
If you prefer to upgrade it to the latest available version, you can use `poetry add package@latest`.

Nothing to add.
(intermediate-level-py3.13)
```

2. Implementación de Módulos

- Descargador Síncrono (src/modulo04/sync_fetcher.py)



```
sync_fetcher.py U X
intermediate_level > src > modulo004 > sync_fetcher.py > fetch_all_sync

1  # src/modulo04/sync_fetcher.py
2  import httpx
3
4
5  def fetch_url_sync(url: str, timeout: float = 5.0) -> dict:
6      """Realiza una petición HTTP GET síncrona."""
7      with httpx.Client(timeout=timeout) as client:
8          response = client.get(url)
9          return {
10             "url": url,
11             "status_code": response.status_code,
12             "size": len(response.content),
13         }
14
15
16  def fetch_all_sync(urls: list[str]) -> list[dict]:
17      """Descarga secuencialmente una lista de URLs."""
18      results = []
19      for url in urls:
20          results.append(fetch_url_sync(url))
21  return results
```

- Descargador Asíncrono con Semáforo (src/modulo04/async_fetcher.py)

El `asyncio.Semaphore` es crucial en producción: limita el número máximo de peticiones para evitar saturar el servidor de destino o agotar sockets locales.

```
sync_fetcher.py U X processor.py U X test_modulo04.py U X async_fetcher.py U X
intermediate_level > src > modulo04 > async_fetcher.py > fetch_all_async
1 # src/modulo04/async_fetcher.py
2 import asyncio
3
4 import httpx
5
6
7 async def fetch_url_async(
8     client: httpx.AsyncClient, url: str, semaphore: asyncio.Semaphore
9 ) -> dict:
10     """Descarga una URL de forma asíncrona respetando el semáforo."""
11     async with semaphore:
12         response = await client.get(url)
13         return {
14             "url": url,
15             "status_code": response.status_code,
16             "size": len(response.content),
17         }
18
19
20 async def fetch_all_async(
21     urls: list[str], max_concurrent: int = 5, timeout: float = 5.0
22 ) -> list[dict]:
23     """Descarga múltiples URLs de forma concurrente usando httpx.AsyncClient y asyncio.Semaphore."""
24     semaphore = asyncio.Semaphore(max_concurrent)
25     async with httpx.AsyncClient(timeout=timeout) as client:
26         tasks = [fetch_url_async(client, url, semaphore) for url in urls]
27         return await asyncio.gather(*tasks)
```

- Procesamiento CPU – Bound (src/modulo04/processor.py)

```
sync_fetcher.py U X processor.py U X test_modulo04.py U X async_fetcher.py U X
intermediate_level > src > modulo04 > processor.py > run_cpu_tasks_parallel
1 # src/modulo04/processor.py
2 from concurrent.futures import ProcessPoolExecutor
3
4
5 def compute_heavy_task(n: int) -> int:
6     """Cálculo CPU-bound intensivo (suma de cuadrados)."""
7     return sum(i * i for i in range(n))
8
9
10 def run_cpu_tasks_parallel(numbers: list[int]) -> list[int]:
11     """Ejecuta múltiples tareas CPU-bound en paralelo aprovechando ProcessPoolExecutor."""
12     with ProcessPoolExecutor() as executor:
13         results = list(executor.map(compute_heavy_task, numbers))
14     return results
```

3. Pruebas Unitarias y Mocks (tests/test_modulo04.py)

```
# tests/test_modulo04.py
import asyncio
from unittest.mock import AsyncMock, MagicMock, patch
```

```

import pytest
from src.modulo04.async_fetcher import fetch_all_async, fetch_url_async
from src.modulo04.processor import compute_heavy_task,
run_cpu_tasks_parallel
from src.modulo04.sync_fetcher import fetch_all_sync, fetch_url_sync

# --- Tests para Fetcher Sincrono ---

@patch("httpx.Client.get")
def test_fetch_url_sync(mock_get):
    mock_response = MagicMock()
    mock_response.status_code = 200
    mock_response.content = b"Hello World"
    mock_get.return_value = mock_response

    result = fetch_url_sync("https://example.com")
    assert result["status_code"] == 200
    assert result["size"] == 11

@patch("src.modulo04.sync_fetcher.fetch_url_sync")
def test_fetch_all_sync(mock_fetch_one):
    mock_fetch_one.return_value = {"url": "https://example.com",
    "status_code": 200, "size": 10}
    urls = ["https://example.com/1", "https://example.com/2"]

    results = fetch_all_sync(urls)
    assert len(results) == 2
    assert mock_fetch_one.call_count == 2

# --- Tests para Fetcher Asíncrono ---

@pytest.mark.asyncio
async def test_fetch_url_async():
    mock_client = AsyncMock()
    mock_response = MagicMock()
    mock_response.status_code = 200
    mock_response.content = b"Async Data"
    mock_client.get.return_value = mock_response

    semaphore = asyncio.Semaphore(2)

```

```

    result = await fetch_url_async(mock_client, "https://example.com",
semaphore)

    assert result["status_code"] == 200
    assert result["size"] == 10

@pytest.mark.asyncio
@patch("httpx.AsyncClient.get")
async def test_fetch_all_async(mock_async_get):
    mock_response = MagicMock()
    mock_response.status_code = 200
    mock_response.content = b"OK"
    mock_async_get.return_value = mock_response

    urls = [f"https://httpbin.org/delay/1?id={i}" for i in range(3)]
    results = await fetch_all_async(urls, max_concurrent=2)

    assert len(results) == 3
    assert all(r["status_code"] == 200 for r in results)

# --- Tests para Procesamiento CPU-Bound ---

def test_compute_heavy_task():
    assert compute_heavy_task(5) == 0 + 1 + 4 + 9 + 16

def test_run_cpu_tasks_parallel():
    numbers = [1000, 2000, 3000]
    results = run_cpu_tasks_parallel(numbers)
    expected = [compute_heavy_task(n) for n in numbers]
    assert results == expected

```

4. Ejecución y Verificación

- Confirmamos la instalación de la dependencia httpx:

```
juan.rivera@mxjuriagdell15400 MINGW64 C:/Users/juan.rivera/Documents/Curso_Python/intermediate_level (feature/modulo-03-pruebas-tdd)
$ poetry add httpx
The following packages are already present in the pyproject.toml and will be skipped:

- httpx

If you want to update it to the latest compatible version, you can use `poetry update package`.
If you prefer to upgrade it to the latest available version, you can use `poetry add package@latest`.

Nothing to add.
(intermediate-level-py3.13)
```

- Corremos la suite de pruebas para verificar la cobertura:

```
$ poetry run pytest --cov=src/modulo04 --cov-report=term-missing -v
===== test session starts =====
platform win32 -- Python 3.13.15, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\juan.rivera\AppData\Local\pypoetry\Cache\virtualenvs\intermediate-level-zUiHL2GR-py3.13\Scripts\python.exe
cachedir: .pytest_cache
hypothesis profile 'default'
rootdir: C:\Users\juan.rivera\Documents\Curso_Python\intermediate_level
configfile: pyproject.toml
plugins: anyio-4.15.1, hypothesis-6.168.1, asyncio-1.4.0, cov-7.1.0
asyncio: mode=Mode.STRICT, debug=False, asyncio_default_fixture_loop_scope=None, asyncio_default_test_loop_scope=function
collected 39 items
```

```
===== warnings summary =====
..\..\AppData\Local\pypoetry\Cache\virtualenvs\intermediate-level-zUiHL2GR-py3.13\Lib\site-packages\fastapi\testclient.py:1
C:\Users\juan.rivera\AppData\Local\pypoetry\Cache\virtualenvs\intermediate-level-zUiHL2GR-py3.13\Lib\site-packages\fastapi\testclient.py:1: StarletteDeprecationWarning: Using 'httpx' with 'starlette.testclient' is deprecated; install 'httpx2' instead.
  from starlette.testclient import TestClient as TestClient # noqa

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
===== tests coverage =====
_____ coverage: platform win32, python 3.13.15-final-0 _____
```

Name	Stmts	Miss	Cover	Missing
src\modulo04\async_fetcher.py	11	0	100%	
src\modulo04\processor.py	7	0	100%	
src\modulo04\sync_fetcher.py	10	0	100%	
TOTAL	28	0	100%	

```
===== 39 passed, 1 warning in 4.14s =====
(intermediate-level-py3.13)
```

- Validamos las métricas de rendimiento esperadas, crearemos un script de benchmarking que contraponga el tiempo de ejecución entre la estrategia síncrona y la asíncrona.

- Script de Benchmark (src/modulo04/benchmark.py)

```
# src/modulo04/benchmark.py
import asyncio
import time

from src.modulo04.async_fetcher import fetch_all_async
from src.modulo04.processor import run_cpu_tasks_parallel
from src.modulo04.sync_fetcher import fetch_all_sync

# Usamos URLs públicas que simulan latencia controlada (delay de 1 segundo por petición)
URLS = [f"https://httpbin.org/delay/1?id={i}" for i in range(5)]
CPU_NUMBERS = [10_000_000, 12_000_000, 15_000_000]
```

```

def run_benchmark():
    print("=== INICIANDO BENCHMARK DEL MÓDULO 04 ===\n")

    # 1. Prueba Síncrona (I/O-bound)
    print(f"1. Descargando {len(URLS)} URLs secuencialmente (Síncrono)...")
    start_sync = time.perf_counter()
    sync_results = fetch_all_sync(URLS)
    duration_sync = time.perf_counter() - start_sync
    print(f"    -> Tiempo Síncrono: {duration_sync:.2f} segundos\n")

    # 2. Prueba Asíncrona (I/O-bound con Semáforo max_concurrent=5)
    print(f"2. Descargando {len(URLS)} URLs concurrentemente (AsyncIO)...")
    start_async = time.perf_counter()
    async_results = asyncio.run(fetch_all_async(URLS,
max_concurrent=5))
    duration_async = time.perf_counter() - start_async
    print(f"    -> Tiempo Asíncrono: {duration_async:.2f} segundos\n")

    # Mejora I/O
    speedup_io = duration_sync / duration_async if duration_async > 0
else 0
    print(f"🚀 [I/O-bound] Mejora de rendimiento: {speedup_io:.2f}x más rápido\n")

    # 3. Prueba CPU-bound (ProcessPoolExecutor)
    print("3. Ejecutando tareas intensivas de CPU en paralelo...")
    start_cpu = time.perf_counter()
    cpu_results = run_cpu_tasks_parallel(CPU_NUMBERS)
    duration_cpu = time.perf_counter() - start_cpu
    print(f"    -> Tiempo de procesamiento CPU (Paralelo): {duration_cpu:.2f} segundos")
    print(f"    -> Resultados obtenidos: {len(cpu_results)} tareas completadas.\n")

    print("BENCHMARK FINALIZADO EN CONCURRENCIA")

if __name__ == "__main__":

```

```
run_benchmark()
```

- Ejecución del Benchmark

```
juan.rivera@mxjuriagdell5400 MINGW64 C:/Users/juan.rivera/Do
$ poetry run python -m src.modulo04.benchmark
INICIANDO BENCHMARK DEL MÓDULO 04

1. Descargando 5 URLs secuencialmente (Síncrono)...
   -> Tiempo Síncrono: 8.17 segundos

2. Descargando 5 URLs concurrentemente (AsyncIO)...
   -> Tiempo Asíncrono: 1.80 segundos

   [I/O-bound] Mejora de rendimiento: 4.53x más rápido

3. Ejecutando tareas intensivas de CPU en paralelo...
   -> Tiempo de procesamiento CPU (Paralelo): 2.19 segundos
   -> Resultados obtenidos: 3 tareas completadas.

BENCHMARK FINALIZADO EN CONCURRENCIA
(intermediate-level-py3.13)
```

Los resultados confirman que:

- I/O-Bound: De 8.17s secuaneciales bajó a 1.80 con asyncio (4.53x mas rapido), optimizando el tiempo de espera de red sin bloquear el hilo principal.
- CPU-Bound: ProcessPoolExecutor distribuyo las 3 tareas intensivas entre los núcleos de nuestro procesador en 2.19s, ejecutándose en paralelo real sin ser frenado por el GIL.

Módulo. Principios SOLID aplicados en Python

Laboratorio:

- Refactor de servicio para depender de puerto (Protocol); implementaciones memoria/SQL y verificación LSP

Arquitectura del laboratorio:

Se diseñará un Servicio de Usuarios (UserService) refactorizado bajo principios SOLID.

- Protocol (Puerto/Contrato) - DIP e ISP:
Definiremos UserRepositoryProtocol mediante typing.Protocol para especificar los métodos exactos de persistencia sin obligar al servicio a depender de detalles de base de datos.
- Implementaciones Concretas – SRP y LSP:
 - InMemoryUserRepository: Implementación ligera sobre diccionarios en memoria (ideal para tests rápidos).
 - SQLiteUserRepository: Implementación real que interactúa con una base de datos SQLite.
 - Ambas cumplirán estrictamente el mismo contrato (LSP) en firmas de métodos, tipos de retorno y manejo de excepciones.
- Inyección de Dependencias y Factory – OCP y DIP:
 - UserService recibirá el repositorio mediante su constructor.
 - Un UserServiceFactory construirá instancias listas para uso en producción o pruebas.

Estructura de archivos:

- src/modulo05/__init__.py
- src/modulo05/contracts.py (Contrato/Protocol)
- src/modulo05/repositories.py (Implementaciones InMemory y SQLite)
- src/modulo05/service.py (Lógica de Negocio y Factory)
- tests/test_modulo05.py (Pruebas unitarias verificando sustitución LSP y comportamiento de ambos repositorios)

1. Protocolo / DIP / ISP (src/modulo05/contracts.py)

Definimos el contrato (puerto) utilizando typing.Protocol. Especifica la interfaz estricta que deben cumplir todos los repositorios de usuarios.

2. Implementaciones & LSP (src/modulo05/repositories.py)

Se implementarán dos repositorios que cumplen con UserRepositoryProtocol:

- InMemoryUserRepository: Guarda datos en un diccionario local.

- SQLiteUserRepository: Guarda datos en un base de datos SQLite en memoria o disco.

Ambas implementaciones mantienen exactamente las mismas firmas y comportamientos esperados para garantizar el Principio de Sustitución de Liskov (LSP):

```
# src/modulo05/repositories.py
import sqlite3

from ..modulo05.contracts import UserDTO

class InMemoryUserRepository:
    """Implementación de persistencia en memoria (para pruebas
    rápidas)."""

    def __init__(self) -> None:
        self._users: dict[str, UserDTO] = {}

    def save(self, user_id: str, email: str, name: str) -> UserDTO:
        user: UserDTO = {"user_id": user_id, "email": email, "name":
name}
        self._users[user_id] = user
        return user

    def find_by_id(self, user_id: str) -> UserDTO | None:
        return self._users.get(user_id)

    def find_by_email(self, email: str) -> UserDTO | None:
        for user in self._users.values():
            if user["email"] == email:
                return user
        return None

class SQLiteUserRepository:
    """Implementación de persistencia real utilizando SQLite."""

    def __init__(self, db_path: str = ":memory:") -> None:
        self.db_path = db_path
        self._init_db()
```

```

def _get_connection(self) -> sqlite3.Connection:
    conn = sqlite3.connect(self.db_path)
    conn.row_factory = sqlite3.Row
    return conn

def _init_db(self) -> None:
    with self._get_connection() as conn:
        conn.execute(
            """
            CREATE TABLE IF NOT EXISTS users (
                user_id TEXT PRIMARY KEY,
                email TEXT UNIQUE NOT NULL,
                name TEXT NOT NULL
            )
            """
        )

def save(self, user_id: str, email: str, name: str) -> UserDTO:
    with self._get_connection() as conn:
        conn.execute(
            "INSERT INTO users (user_id, email, name) VALUES (?, ?, ?)",
            (user_id, email, name),
        )
    return {"user_id": user_id, "email": email, "name": name}

def find_by_id(self, user_id: str) -> UserDTO | None:
    with self._get_connection() as conn:
        cursor = conn.execute(
            "SELECT user_id, email, name FROM users WHERE user_id = ?",
            (user_id,),
        )
        row = cursor.fetchone()
        if row:
            return {
                "user_id": row["user_id"],
                "email": row["email"],
                "name": row["name"],
            }
    return None

```

```

def find_by_email(self, email: str) -> UserDTO | None:
    with self._get_connection() as conn:
        cursor = conn.execute(
            "SELECT user_id, email, name FROM users WHERE email =
?",
            (email,),
        )
        row = cursor.fetchone()
        if row:
            return {
                "user_id": row["user_id"],
                "email": row["email"],
                "name": row["name"],
            }
        return None

```

3. Lógica de Negocio y Factory (src/modulo05/service.py)

UserService implementa la lógica de negocio y depende exclusivamente de UserRepositoryProtocol (Inversión de Dependencias). Además, implementamos UserServiceFactory para simplificar la creación del servicio según el entorno.

4. Verificación LSP y Pruebas Unitarias (test/test_modulo05.py)

Diseñamos pruebas parametrizadas utilizando `pytest.mark.parametrize` para inyectar tanto `InMemoryUserRepository` como `SQLiteUserRepository`. Esto verificará explícitamente el Principio de Sustitución de Liskov (LSP): ambas implementaciones deben responder de forma idéntica ante los mismos estímulos.

```

# tests/test_modulo05.py
import pytest
from src.modulo05.repositories import InMemoryUserRepository,
SQLiteUserRepository
from src.modulo05.service import UserService, UserServiceFactory

@pytest.fixture(
    params=[
        InMemoryUserRepository,

```

```

        lambda: SQLiteUserRepository(":memory:"),
    ]
)

def repository_instance(request):
    """Fixture parametrizada que retorna ambas implementaciones de
    repositorio.

    Permite comprobar de forma automatizada que ambas respetan LSP.
    """
    factory = request.param
    return factory()

def test_lsp_repository_implementations_behavior(repository_instance):
    """Verifica el cumplimiento de LSP: ambas implementaciones
    responden igual."""
    service = UserService(repository=repository_instance)

    # 1. Registro exitoso
    user = service.register_user("juan@example.com", "Juan Rivera")
    assert user["email"] == "juan@example.com"
    assert user["name"] == "Juan Rivera"
    assert "user_id" in user

    # 2. Búsqueda por ID
    found_user = service.get_user_by_id(user["user_id"])
    assert found_user == user

    # 3. Prevenir duplicados por email
    with pytest.raises(ValueError, match="ya existe"):
        service.register_user("juan@example.com", "Otro Nombre")

def test_register_user_invalid_email():
    service = UserServiceFactory.create_for_testing()
    with pytest.raises(ValueError, match="Formato de email inválido"):
        service.register_user("invalid-email", "Juan")

def test_register_user_empty_name():
    service = UserServiceFactory.create_for_testing()

```

```

    with pytest.raises(ValueError, match="El nombre no puede estar vacío"):
        service.register_user("juan@example.com", " ")

def test_get_user_by_id_not_found():
    service = UserServiceFactory.create_for_testing()
    with pytest.raises(KeyError, match="no encontrado"):
        service.get_user_by_id("non-existent-id")

def test_user_service_factory_production(tmp_path):
    db_file = str(tmp_path / "test_prod.db")
    service = UserServiceFactory.create_for_production(db_path=db_file)
    user = service.register_user("prod@example.com", "Prod User")
    assert user["email"] == "prod@example.com"

```

5. Verificación con Pytest:

Ejecutamos la suite de pruebas completa en la terminal con Poetry:

```

juan.rivera@mxjuniagdel15400 MINGW64 C:\Users\juan.rivera\Documents\Curso_Python\intermediate_level (feature/modulo-05-principios-solid)
$ poetry run pytest --cov=src/modulo05 --cov-report=term-missing -v
===== test session starts =====
platform win32 -- Python 3.13.15, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\juan.rivera\AppData\Local\pypoetry\Cache\virtualenvs\intermediate-level-zUHL2GR-py3.13\Scripts\python.exe
cachedir: .pytest_cache
hypothesis profile 'default'
rootdir: C:\Users\juan.rivera\Documents\Curso_Python\intermediate_level
configfile: pyproject.toml
plugins: anyio-4.15.1, hypothesis-6.168.1, asyncio-1.4.0, cov-7.1.0
asyncio: mode=Mode.STRICT, debug=False, asyncio_default_fixture_loop_scope=None, asyncio_default_test_loop_scope=function
collected 11 items

```

```

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
===== tests coverage =====
_____ coverage: platform win32, python 3.13.15-final-0 _____

Name                                Stmts   Miss  Cover   Missing
-----
src\modulo05\__init__.py              0      0   100%
src\modulo05\contracts.py             6      0   100%
src\modulo05\repositories.py          41      1    98%    69
src\modulo05\service.py              30      0   100%
-----
TOTAL                                77      1    99%
===== 11 passed, 3 warnings in 2.81s =====
(intermediate-level-py3.13)

```

Módulo. Patrones de diseño

Laboratorio:

- Strategy para precios; Decorator de caché; Adapter para proveedor externo; pruebas con pytest

Arquitectura del Laboratorio:

En este laboratorio se construirá un sistema de procesamiento de precios y pagos aplicando patrones idiomáticos de Python:

- Patrón Strategy (Precios y Descuento):
Se utilizaron Callables / funciones como estrategias intercambiables para calcular precios finales sin crear jerarquías de clases redundantes.
- Patrón Decorator (Caché e Inspección):
Se creó un decorator `memoize_cache` que agregará soporte de almacenamiento en caché transparente sobre métodos o funciones costosas.

- Patrón Adaptar (Proveedor Externo):
Se adaptó un servicio de pago externo de terceros con un formato incompatible (LegacyPaymentGateway) a nuestra interfaz/protocolo interno (PaymentProcessorProtocol).

Estructura de archivos:

- src/modulo06/__init__.py
- src/modulo06/strategies.py (Estrategias de descuento – Strategy)
- src/modulo06/decorators.py (Decorator de caché - Decorator)
- src/modulo06/adapters.py (Adaptador para proveedor legacy – Adapter)
- src/modulo06/service.py (Servicio integrador)
- tests/test_modulo06.py (Pruebas unitarias de los tres patrones con pytest)

1. Módulo de Estrategias (src/modulo06/strategies.py)

```
from collections.abc import Callable
from typing import Protocol

class DiscountStrategyProtocol(Protocol):
    """Protocolo opcional para tipar explícitamente las estrategias de
    descuento."""

    def __call__(self, price: float) -> float: ...

def no_discount(price: float) -> float:
    """Estrategia por defecto: sin descuento."""
    return price

def percentage_discount(percentage: float) -> Callable[[float], float]:
    """Closure fábrica de estrategia para aplicar un porcentaje de
    descuento."""
    if not (0 <= percentage <= 100):
        raise ValueError("El porcentaje debe estar entre 0 y 100")

    def strategy(price: float) -> float:
        return price * (1 - percentage / 100)

    return strategy
```

```
def fixed_amount_discount(amount: float) -> Callable[[float], float]:
    """Closure fábrica de estrategia para aplicar un descuento fijo en
    monto."""

    def strategy(price: float) -> float:
        return max(0.0, price - amount)

    return strategy
```

2. Decorador de Caché (src/modulo06/decorators.py)

Se implementa un decorador con memoization para cachear llamadas a funciones costosas.

```
from collections.abc import Callable
from functools import wraps
from typing import Any, ParamSpec, TypeVar

P = ParamSpec("P")
R = TypeVar("R")

def memoize_cache(func: Callable[P, R]) -> Callable[P, R]:
    """Decorador estructural para almacenar en caché resultados de
    llamadas a funciones."""
    cache: dict[tuple[Any, ...], R] = {}

    @wraps(func)
    def wrapper(*args: P.args, **kwargs: P.kwargs) -> R:
        key = (args, tuple(sorted(kwargs.items())))
        if key not in cache:
            cache[key] = func(*args, **kwargs)
        return cache[key]

    wrapper.cache = cache # type: ignore[attr-defined]
    return wrapper
```

3. Adaptador de Pagos (src/modulo06/adapters.py)

Se aplicará el patrón Adapter. Nos imaginamos que tenemos una biblioteca o SDK legacy (LegacyPaymentGateway) que espera montos en centavos y devuelve un diccionario, pero nuestro sistema moderno espera una interfaz uniforme con booleanos y montos en unidades completa.

```

from typing import Protocol

class PaymentProcessorProtocol(Protocol):
    """Interfaz estándar que nuestro sistema espera."""

    def process_payment(self, amount: float) -> bool: ...

class LegacyPaymentGateway:
    """Clase externa/heredada con una interfaz incompatible."""

    def make_transaction(
        self, cost_in_cents: int, currency: str = "USD"
    ) -> dict[str, str | int]:
        if cost_in_cents <= 0:
            return {"status": "FAILED", "code": 400}
        return {"status": "SUCCESS", "code": 200}

class LegacyPaymentAdapter:
    """Adaptador que convierte la interfaz de LegacyPaymentGateway a
    PaymentProcessorProtocol."""

    def __init__(self, legacy_gateway: LegacyPaymentGateway) -> None:
        self._legacy_gateway = legacy_gateway

    def process_payment(self, amount: float) -> bool:
        # Conversión de pesos/dólares a centavos requeridos por el
servicio legacy
        amount_in_cents = int(amount * 100)
        response =
self._legacy_gateway.make_transaction(amount_in_cents)
        return response.get("status") == "SUCCESS"

```

4. Servicio Integrador (src/modulo06/service.py)

Se crea la clase CheckoutService, la cual coordinará la aplicación de las estrategias de descuento, la ejecución con caché del cálculo de impuestos y el envío del cobro mediante el adaptador.

```

from collections.abc import Callable

from ..modulo06.adapters import PaymentProcessorProtocol
from ..modulo06.decorators import memoize_cache
from ..modulo06.strategies import no_discount

class CheckoutService:
    """Servicio que integra las estrategias de precio, caché y
    procesamiento de pago."""

    def __init__(self, payment_processor: PaymentProcessorProtocol) ->
None:
        self.payment_processor = payment_processor

    @memoize_cache
    def calculate_tax(self, amount: float, tax_rate: float = 0.16) ->
float:
        """Simulación de un cálculo costoso protegido por el decorador
        de caché."""
        return round(amount * tax_rate, 2)

    def checkout(
        self,
        base_price: float,
        discount_strategy: Callable[[float], float] = no_discount,
    ) -> bool:
        discounted_price = discount_strategy(base_price)
        tax = self.calculate_tax(discounted_price)
        total = discounted_price + tax
        return self.payment_processor.process_payment(total)

```

5. Pruebas Unitarias (tests/test_modulo06.py)

Se crea la suite de pruebas unitarias para validar que cada patrón funciona individualmente y en conjunto dentro del servicio.

```

import pytest

from src.modulo06.adapters import LegacyPaymentAdapter,
LegacyPaymentGateway
from src.modulo06.service import CheckoutService

```

```

from src.modulo06.strategies import (
    fixed_amount_discount,
    no_discount,
    percentage_discount,
)

def test_strategy_discounts():
    price = 100.0

    assert no_discount(price) == 100.0
    assert percentage_discount(20)(price) == 80.0
    assert fixed_amount_discount(15)(price) == 85.0

def test_strategy_invalid_percentage():
    with pytest.raises(ValueError):
        percentage_discount(150)

def test_adapter_integration():
    legacy_sdk = LegacyPaymentGateway()
    adapter = LegacyPaymentAdapter(legacy_sdk)

    assert adapter.process_payment(50.0) is True
    assert adapter.process_payment(-10.0) is False

def test_checkout_service_with_cache_and_adapter():
    legacy_sdk = LegacyPaymentGateway()
    adapter = LegacyPaymentAdapter(legacy_sdk)
    service = CheckoutService(payment_processor=adapter)

    # 1. Probar Checkout exitoso usando estrategia de 10% descuento
    success = service.checkout(
        base_price=100.0, discount_strategy=percentage_discount(10)
    )
    assert success is True

    # 2. Verificar funcionamiento del decorador de caché en
    calculate_tax
    tax1 = service.calculate_tax(90.0)

```

```

tax2 = service.calculate_tax(90.0)

assert tax1 == 14.4
assert tax1 == tax2
# Verificar que el resultado provino de la caché inspeccionando el
dict del decorador
assert len(service.calculate_tax.cache) == 1 # type: ignore[attr-
defined]

```

6. Verificación

Ejecutamos nuestro checklist obligatorio de verificación:

```

juan.rivera@mxjuriagdel15480 MINGW64 C:/Users/juan.rivera/Documents/Curso_Python/intermediate_level (feature/modulo-06-patrones-diseno)
$ poetry run pytest tests/test_modulo06.py -v
===== test session starts =====
platform win32 -- Python 3.13.15, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\juan.rivera\AppData\Local\pypoetry\Cache\virtualenvs\intermediate-level-zUiHL2GR-py3.
13\Scripts\python.exe
cachedir: .pytest_cache
hypothesis profile 'default'
rootdir: C:/Users/juan.rivera/Documents/Curso_Python/intermediate_level
configfile: pyproject.toml
plugins: anyio-4.15.1, hypothesis-6.168.1, asyncio-1.4.0, cov-7.1.0
asyncio: mode=Mode.STRICT, debug=False, asyncio_default_fixture_loop_scope=None, asyncio_default_test_loop_scope=function
collected 4 items

tests/test_modulo06.py::test_strategy_discounts PASSED [ 25%]
tests/test_modulo06.py::test_strategy_invalid_percentage PASSED [ 50%]
tests/test_modulo06.py::test_adapter_integration PASSED [ 75%]
tests/test_modulo06.py::test_checkout_service_with_cache_and_adapter PASSED [100%]

===== 4 passed in 0.23s =====

```

Módulo. Ciencia de datos

Laboratorio:

Cargar/limpiar CSV en Pandas; entrenar un clasificador; guardar con joblib y probar inferencia

1. Crear un dataset CSV sintético

Para cumplir con la lectura desde CSV usando Pandas, se creará una función que asegure la existencia de un archivo CSV local

```

from pathlib import Path

import numpy as np
import pandas as pd

def create_sample_csv(file_path: Path = Path("data/dataset.csv")) ->
None:

```

```

"""Genera un archivo CSV sintético si no existe."""
if file_path.exists():
    return

file_path.parent.mkdir(parents=True, exist_ok=True)
np.random.seed(42)
num_samples = 500

amount = np.random.uniform(10.0, 5000.0, size=num_samples)
transaction_count = np.random.randint(1, 50, size=num_samples)
account_age_days = np.random.randint(30, 3650, size=num_samples)

# Introducir algunos valores nulos para ejercitar la limpieza con
Pandas
amount[:, :20] = np.nan

risk_score = (
    (np.nan_to_num(amount) / 5000.0) * 0.5
    + (transaction_count / 50.0) * 0.3
    - (account_age_days / 3650.0) * 0.2
)
is_high_risk = (risk_score > 0.4).astype(int)

df = pd.DataFrame(
    {
        "amount": amount,
        "transaction_count": transaction_count,
        "account_age_days": account_age_days,
        "is_high_risk": is_high_risk,
    }
)
df.to_csv(file_path, index=False)

def load_and_clean_data(
    file_path: Path = Path("data/dataset.csv"),
) -> tuple[pd.DataFrame, pd.Series]:
    """Carga y limpia un dataset CSV usando Pandas."""
    if not file_path.exists():
        create_sample_csv(file_path)

# 1. Cargar CSV en Pandas

```

```

df = pd.read_csv(file_path)

# 2. Limpieza de datos (eliminar filas con valores nulos o
rellenar)
df_clean = df.dropna().copy()

# 3. Separar características (X) y variable objetivo (y)
x_features = df_clean[["amount", "transaction_count",
"account_age_days"]]
y_target = df_clean["is_high_risk"]

return x_features, y_target

```

2. Entrenar el clasificador y guardar con joblib (src/modulo07/model_pipeline.py)

Se utilizará Pandas y scikit-learn para entrenar un clasificador (LogisticRegression o RandomForestClassifier), evaluarlo y serializarlo en un archivo .joblib.

```

from pathlib import Path

import joblib
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
from sklearn.model_selection import train_test_split

from src.modulo07.data_processing import load_and_clean_data

def train_and_save_classifier(
    data_path: Path = Path("data/dataset.csv"),
    model_path: Path = Path("models/classifier.joblib"),
) -> float:
    """Carga datos limpios, entrena un clasificador y lo guarda con
    joblib."""
    x_data, y_data = load_and_clean_data(data_path)

    # Dividir en entrenamiento (80%) y prueba (20%)
    x_train, x_test, y_train, y_test = train_test_split(
        x_data, y_data, test_size=0.2, random_state=42
    )

```

```

    )

    # Entrenar clasificador
    classifier = RandomForestClassifier(n_estimators=50,
random_state=42, n_jobs=1)
    classifier.fit(x_train, y_train)

    # Evaluar desempeño
    y_pred = classifier.predict(x_test)
    accuracy = float(accuracy_score(y_test, y_pred))

    # Guardar modelo entrenado con joblib
    model_path.parent.mkdir(parents=True, exist_ok=True)
    joblib.dump(classifier, model_path)

    return accuracy

if __name__ == "__main__":
    acc = train_and_save_classifier()
    print(f"Clasificador entrenado y guardado. Precisión: {acc:.4f}")

```

3. Exponer y probar la inferencia mínima (src/modulo07/inference.py)

Creemos el archivo src/modulo07/inference.py para cargar el modelo guardado por joblib y realizar predicciones con nuevos datos recibidos:

```

from pathlib import Path
from typing import Any

import joblib
import pandas as pd

class MinimumInferenceService:
    """Servicio para cargar un modelo guardado y ejecutar inferencias
sobre datos de entrada."""

```

```

def __init__(self, model_path: Path =
Path("models/classifier.joblib")):
    self.model_path = model_path
    self._model: Any = None

def load_model(self) -> None:
    """Carga el clasificador serializado (.joblib)."""
    if not self.model_path.exists():
        raise FileNotFoundError(
            f"No se encontró el modelo en la ruta:
{self.model_path}"
        )
    self._model = joblib.load(self.model_path)

def predict(
    self, amount: float, transaction_count: int, account_age_days:
int
) -> int:
    """Realiza la inferencia utilizando un DataFrame de Pandas de
una sola fila."""
    if self._model is None:
        self.load_model()

    input_df = pd.DataFrame(
        [
            {
                "amount": amount,
                "transaction_count": transaction_count,
                "account_age_days": account_age_days,
            }
        ]
    )
    prediction = self._model.predict(input_df)[0]
    return int(prediction)

```

4. Pruebas Unitarias (test/test_modulo07.py)

Creamos la suite de pruebas para validar todo el flujo del laboratorio (cargar CSV con Pandas, limpiar, entrenar, guardar con joblib e inferir)

```

from pathlib import Path

```

```

import pandas as pd

from src.modulo07.data_processing import load_and_clean_data
from src.modulo07.inference import MinimumInferenceService
from src.modulo07.model_pipeline import train_and_save_classifier

def test_load_and_clean_data(tmp_path: Path):
    csv_path = tmp_path / "test_dataset.csv"
    x_features, y_target = load_and_clean_data(csv_path)

    assert isinstance(x_features, pd.DataFrame)
    assert isinstance(y_target, pd.Series)
    assert not x_features.isnull().values.any()
    assert len(x_features) == len(y_target)

def test_train_save_and_inference(tmp_path: Path):
    csv_path = tmp_path / "data.csv"
    model_path = tmp_path / "model.joblib"

    # 1. Entrenar y guardar con joblib
    acc = train_and_save_classifier(data_path=csv_path,
model_path=model_path)
    assert acc >= 0.70
    assert model_path.exists()

    # 2. Probar inferencia mínima
    service = MinimumInferenceService(model_path=model_path)
    prediction = service.predict(
        amount=2500.0, transaction_count=30, account_age_days=180
    )
    assert prediction in (0, 1)

```

5. Ejecutar verificación:

```

juan.rivera@mxjuriagdell5400 MINGW64 C:/Users/juan.rivera/Documents/Curso_Python/intermediate_level (feature/modulo-07-ciencia-datos)
$ poetry run ruff check .
All checks passed!
(intermediate-level-py3.13)
juan.rivera@mxjuriagdell5400 MINGW64 C:/Users/juan.rivera/Documents/Curso_Python/intermediate_level (feature/modulo-07-ciencia-datos)
$ poetry run ruff check . --fix
All checks passed!
(intermediate-level-py3.13)
juan.rivera@mxjuriagdell5400 MINGW64 C:/Users/juan.rivera/Documents/Curso_Python/intermediate_level (feature/modulo-07-ciencia-datos)
$ poetry run ruff format .
51 files left unchanged
(intermediate-level-py3.13)
juan.rivera@mxjuriagdell5400 MINGW64 C:/Users/juan.rivera/Documents/Curso_Python/intermediate_level (feature/modulo-07-ciencia-datos)
$ poetry run mypy src
Success: no issues found in 38 source files
(intermediate-level-py3.13)

```

```

juan.rivera@mxjuriagdell5400 MINGW64 C:/Users/juan.rivera/Documents/Curso_Python/intermediate_level (feature/modulo-07-ciencia-datos)
$ OMP_NUM_THREADS=1 poetry run pytest
===== test session starts =====
platform win32 -- Python 3.13.15, pytest-9.1.1, pluggy-1.6.0
rootdir: C:\Users\juan.rivera\Documents\Curso_Python\intermediate_level
configfile: pyproject.toml
plugins: anyio-4.15.1, hypothesis-6.168.1, asyncio-1.4.0, cov-7.1.0
asyncio: mode=Mode.STRICT, debug=False, asyncio_default_fixture_loop_scope=None, asyncio_default_test_loop_scope=function
collected 51 items

tests\test_modulo02.py .. [ 3%]
tests\test_modulo04.py ..... [ 15%]
tests\test_modulo05.py ..... [ 27%]
tests\test_modulo06.py .... [ 35%]
tests\test_modulo07.py .. [ 39%]
tests\test_modulo1.py ... [ 45%]
tests\test_payment_processor.py ..... [ 70%]
tests\test_wallet.py ..... [100%]

===== warnings summary =====
..\..\AppData\Local\pypoetry\Cache\virtualenvs\intermediate-level-zUiHL2GR-py3.13\Lib\site-packages\fastapi\testclient.py:1
C:\Users\juan.rivera\AppData\Local\pypoetry\Cache\virtualenvs\intermediate-level-zUiHL2GR-py3.13\Lib\site-packages\fastapi\testclient.py:1: StarletteDeprecationWarning: Using 'httpx' with 'starlette.testclient' is deprecated; install 'httpx2' instead.
from starlette.testclient import TestClient as TestClient # noqa

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
===== 51 passed, 1 warning in 25.62s =====

```

Módulo. Arquitectura Hexagonal (Puerto y Adaptadores)

Laboratorio:

- Caso de uso CreateOrder con puerto de repositorio y adaptadores en memoria/SQLAlchemy; adaptador de notificación HTTP simulado; pruebas de contrato

En este laboratorio se implementará:

- Entidad Order y sus reglas de negocio (Dominio).
- Puertos de Repositorio de Ordenes y Notificar HTTP junto al caso de uso CreateOrder (Aplicación).
- Adaptadores en Memoria y SQLAlchemy para la persistencia, más el adaptador de notificación HTTP simulado (Infraestructura).
- Pruebas de Contrato y Dominio para validar el comportamiento intercambiable de los adaptadores (Tests).

1. Dominio – Entidad Order y Excepciones (src/modulo08/domain/order.py)

Definimos la entidad de dominio Order con sus reglas de negocio (monto positivo, items requeridos, estados de la orden)

```

import uuid
from dataclasses import dataclass, field
from enum import Enum

from ...modulo08.domain.exceptions import DomainError

class InvalidOrderError(DomainError):
    """Excepción cuando una orden no cumple con las reglas del
    negocio."""

class OrderStatus(str, Enum):
    CREATED = "CREATED"
    COMPLETED = "COMPLETED"
    CANCELLED = "CANCELLED"

@dataclass
class OrderItem:
    product_id: str
    quantity: int
    unit_price: float

    def __post_init__(self) -> None:
        if self.quantity <= 0:
            raise InvalidOrderError("La cantidad del producto debe ser
mayor a cero.")
        if self.unit_price <= 0:
            raise InvalidOrderError("El precio unitario debe ser mayor
a cero.")

    @property
    def total_price(self) -> float:
        return self.quantity * self.unit_price

@dataclass
class Order:
    order_id: str
    customer_id: str

```

```

items: list[OrderItem] = field(default_factory=list)
status: OrderStatus = OrderStatus.CREATED

def __post_init__(self) -> None:
    if not self.customer_id.strip():
        raise InvalidOrderError("El ID del cliente no puede estar vacío.")
    if not self.items:
        raise InvalidOrderError("Una orden debe tener al menos un ítem.")

    @classmethod
    def create(cls, customer_id: str, items: list[OrderItem]) -> "Order":
        return cls(
            order_id=str(uuid.uuid4()),
            customer_id=customer_id,
            items=items,
            status=OrderStatus.CREATED,
        )

    @property
    def total_amount(self) -> float:
        return sum(item.total_price for item in self.items)

```

2. Capa de Aplicación - DTOs, Puertos y Caso de Uso CreateOrder

- DTOs de Órdenes (src/modulo08/aplication/order_dtos.py)

```

from dataclasses import dataclass

@dataclass(frozen=True)
class OrderItemDTO:
    """DTO de entrada/salida para un ítem de la orden."""

    product_id: str
    quantity: int
    unit_price: float

@dataclass(frozen=True)

```

```

class CreateOrderDTO:
    """DTO de entrada para crear una orden."""

    customer_id: str
    items: list[OrderItemDTO]

@dataclass(frozen=True)
class OrderResponseDTO:
    """DTO de salida con los detalles de la orden creada."""

    order_id: str
    customer_id: str
    total_amount: float
    status: str

```

- Puertos de Órdenes (src/modulo08/application/order_ports.py)

Se definen los contratos para el repositorio de órdenes y el notificador HTTP simulado:

```

from typing import Protocol

from ...modulo08.domain.order import Order

class OrderRepositoryPort(Protocol):
    """Puerto para la persistencia de órdenes."""

    def save(self, order: Order) -> None:
        ...

    def get_by_id(self, order_id: str) -> Order | None:
        ...

class HTTPNotifierPort(Protocol):
    """Puerto para el envío de notificaciones HTTP externas."""

    def notify_order_created(
        self, order_id: str, customer_id: str, total_amount: float
    ) -> None:
        ...

```

```
) -> bool:  
    ...
```

- Caso de Uso CreateOrder
(src/modulo08/application/createorderuse_case.py)

```
from ...modulo08.application.order_dtos import (  
    CreateOrderDTO,  
    OrderResponseDTO,  
)  
from ...modulo08.application.order_ports import (  
    HTTPNotifierPort,  
    OrderRepositoryPort,  
)  
from ...modulo08.domain.order import Order, OrderItem  
  
class CreateOrderUseCase:  
    """Caso de uso: Crear e ingresar una nueva orden en el sistema."""  
  
    def __init__(  
        self,  
        repository: OrderRepositoryPort,  
        notifier: HTTPNotifierPort,  
    ) -> None:  
        self.repository = repository  
        self.notifier = notifier  
  
    def execute(self, dto: CreateOrderDTO) -> OrderResponseDTO:  
        # 1. Convertir DTOs a entidades de dominio  
        items = [  
            OrderItem(  
                product_id=item_dto.product_id,  
                quantity=item_dto.quantity,  
                unit_price=item_dto.unit_price,  
            )  
            for item_dto in dto.items  
        ]
```

```

# 2. Instanciar entidad y aplicar reglas de negocio
order = Order.create(customer_id=dto.customer_id, items=items)

# 3. Persistir usando el puerto
self.repository.save(order)

# 4. Notificar mediante el puerto HTTP
self.notifier.notify_order_created(
    order_id=order.order_id,
    customer_id=order.customer_id,
    total_amount=order.total_amount,
)

# 5. Retornar DTO de respuesta
return OrderResponseDTO(
    order_id=order.order_id,
    customer_id=order.customer_id,
    total_amount=order.total_amount,
    status=order.status.value,
)

```

3. Adaptadores de Infraestructura (src/modulo08/infraestructura/adapters/)

Aquí se crearán los dos adaptadores de persistencia intercambiables y el adaptador para la notificación HTTP.

- Adaptador en Memoria
(src/modulo08/infraestructura/adapters/orderinmemory_db.py)

```

from ....modulo08.application.order_ports import OrderRepositoryPort
from ....modulo08.domain.order import Order

class InMemoryOrderRepositoryAdapter(OrderRepositoryPort):
    """Adaptador de infraestructura: Almacenamiento de órdenes en
    memoria."""

```

```

def __init__(self) -> None:
    self._storage: dict[str, Order] = {}

def save(self, order: Order) -> None:
    self._storage[order.order_id] = order

def get_by_id(self, order_id: str) -> Order | None:
    return self._storage.get(order_id)

```

- Adaptador SQLAlchemy
(src/modulo08/infrastructure/adapters/order_sqlalchemy_db.py)

Se implementa OrderRepositoryPort mapeandp entre modelos de ORM y Entidades de Dominio:

```

from sqlalchemy import Column, Float, Integer, String
from sqlalchemy.orm import DeclarativeBase, Session, sessionmaker

from ....modulo08.application.order_ports import OrderRepositoryPort
from ....modulo08.domain.order import Order, OrderItem, OrderStatus

class Base(DeclarativeBase):
    pass

class OrderModel(Base):
    __tablename__ = "orders"

    order_id = Column(String, primary_key=True)
    customer_id = Column(String, nullable=False)
    status = Column(String, nullable=False)

class OrderItemModel(Base):
    __tablename__ = "order_items"

    id = Column(Integer, primary_key=True, autoincrement=True)
    order_id = Column(String, nullable=False)
    product_id = Column(String, nullable=False)

```

```

quantity = Column(Integer, nullable=False)
unit_price = Column(Float, nullable=False)

class SQLAlchemyOrderRepositoryAdapter(OrderRepositoryPort):
    """Adaptador de infraestructura: Base de datos SQL vía
    SQLAlchemy."""

    def __init__(self, session_factory: sessionmaker[Session]) -> None:
        self.session_factory = session_factory

    def save(self, order: Order) -> None:
        with self.session_factory() as session:
            # Insertar o actualizar la orden principal
            order_record = OrderModel(
                order_id=order.order_id,
                customer_id=order.customer_id,
                status=order.status.value,
            )
            session.merge(order_record)

            # Insertar los ítems asociados
            for item in order.items:
                item_record = OrderItemModel(
                    order_id=order.order_id,
                    product_id=item.product_id,
                    quantity=item.quantity,
                    unit_price=item.unit_price,
                )
                session.add(item_record)

            session.commit()

    def get_by_id(self, order_id: str) -> Order | None:
        with self.session_factory() as session:
            order_record = (
                session.query(OrderModel)
                .filter(OrderModel.order_id == order_id)
                .first()
            )
            if order_record is None:
                return None

```

```

        item_records = (
            session.query(OrderItemModel)
                .filter(OrderItemModel.order_id == order_id)
                .all()
        )

        items = [
            OrderItem(
                product_id=rec.product_id,
                quantity=rec.quantity,
                unit_price=rec.unit_price,
            )
            for rec in item_records
        ]

        return Order(
            order_id=order_record.order_id,
            customer_id=order_record.customer_id,
            items=items,
            status=OrderStatus(order_record.status),
        )

```

- Adaptador HTTP Simulado
(src/modulo08/infrastructure/adapters/http_notifies.py)

```

from ...modulo08.application.order_ports import HTTPNotifierPort

class MockHTTPNotifierAdapter(HTTPNotifierPort):
    """Adaptador de infraestructura: Notificador HTTP simulado para pruebas."""

    def __init__(self) -> None:
        self.requests_log: list[dict[str, str | float]] = []

    def notify_order_created(
        self, order_id: str, customer_id: str, total_amount: float
    ):

```

```

) -> bool:
    payload = {
        "order_id": order_id,
        "customer_id": customer_id,
        "total_amount": total_amount,
    }
    self.requests_log.append(payload)
    return True

```

4. Pruebas de Contrato y Dominio (tests/test_modulo08_lab.py)

```

(intermediate-level-py3.13)
juan.rivera@mxjuriagdell5400 MINGW64 C:/Users/juan.rivera/Documents/Curso_Python/intermediate_level (feature/modulo-08-arquitectura-hexagonal)
$ poetry run mypy src
Success: no issues found in 60 source files
(intermediate-level-py3.13)
juan.rivera@mxjuriagdell5400 MINGW64 C:/Users/juan.rivera/Documents/Curso_Python/intermediate_level (feature/modulo-08-arquitectura-hexagonal)
$ OMP_NUM_THREADS=1 poetry run pytest
===== test session starts =====
platform win32 -- Python 3.13.15, pytest-9.1.1, pluggy-1.6.0
rootdir: C:\Users\juan.rivera\Documents\Curso_Python\intermediate_level
configfile: pyproject.toml
plugins: anyio-4.15.1, hypothesis-6.168.1, asyncio-1.4.0, cov-7.1.0
asyncio: mode=Mode.STRICT, debug=False, asyncio_default_fixture_loop_scope=None, asyncio_default_test_loop_scope=function
collected 62 items

tests\test_modulo02.py .. [ 3%]
tests\test_modulo04.py ..... [ 12%]
tests\test_modulo05.py ..... [ 22%]
tests\test_modulo06.py .... [ 29%]
tests\test_modulo07.py .. [ 32%]
tests\test_modulo08.py ..... [ 40%]
tests\test_modulo08_lab.py ..... [ 50%]
tests\test_modulo01.py ... [ 54%]
tests\test_payment_processor.py ..... [ 75%]
tests\test_wallet.py ..... [100%]

===== warnings summary =====
..\..\AppData\Local\pypoetry\Cache\virtualenvs\intermediate-level-zUiHL2GR-py3.13\Lib\site-packages\fastapi\testclient.py:1
C:\Users\juan.rivera\AppData\Local\pypoetry\Cache\virtualenvs\intermediate-level-zUiHL2GR-py3.13\Lib\site-packages\fastapi\testclient.py:1: StarletteDeprecationWarning: Using 'httpx' with 'starlette.testclient' is deprecated; install 'httpx2' instead.

```